

Agentic AI 기반 클라우드 비용 이상 징후 탐지 및 자율 복구 시스템



202355514 강지원 kjw497@pusan.ac.kr

202355540 박소영 qkrthdud1309@pusan.ac.kr

202355594 허소영 thdual429@pusan.ac.kr

지도교수 김태운

목 차

1. 요약.....	1
2. 서론.....	1
2.1. 연구 배경.....	1
2.2. 기존 문제점	2
2.2.1. 수동 모니터링 대응의 한계.....	2
2.2.2. 기존 FinOps 도구들의 한계 및 차별점	3
2.3. 연구 목표.....	3
2.4. 기대 효과	4
3. 연구 배경.....	5
3.1. 배경 지식.....	5
3.1.1. 클라우드 비용 이상.....	5
3.1.2. 멀티에이전트 파이프라인	7
3.1.3. 클라우드.....	8
3.1.4. FinOps.....	8
3.1.5. Isolation Forest.....	9
3.1.6. Z-Score.....	9
3.1.7. 이상탐지 검증 지표: Clopper-Pearson 신뢰구간	10
3.1.8. LangGraph	10
3.1.9. Human-in-the-loop (HITL)와 승인 기반 자동화	11
3.1.10. WAF와 Rate-based Rule.....	12
3.1.11. SLA와 가용성 지표.....	13

4. 연구 내용	14
4.1. 시스템 아키텍처 및 기술 스택	14
4.1.1. 전체 아키텍처 개요.....	14
4.1.2. 기술 스택 선정 근거	15
4.2. 에이전트	16
4.2.1. Detection Agent.....	16
4.2.2. Classification Agent.....	17
4.2.3. Decision Agent	17
4.2.4. Action Agent.....	19
4.2.5. QA Agent	19
4.2.6. Logging Agent.....	20
4.3. 이상 탐지 모델 설계 (Isolation Forest / Z-score / 고정임계값).....	21
4.3.1. 데이터 준비.....	21
4.3.2. 데이터 분석 및 하이퍼파라미터 값 선정	21
4.3.3. 모델 학습용 데이터셋 구축 및 평가	22
4.3.4. 최적화 (성능 최적화, 정확도 최적화 등 디버깅 과정)	24
4.4. 자동화 대응 및 운영 시스템	25
4.4.1. 탐지-대응 파이프라인 (inbound 규칙, rule-book 도입)	25
4.4.2. 가중치 조절 메커니즘 (관리자 설정 → LLM 프롬프트 동적 반영)	27
4.4.3. 승인 대기 메커니즘 (LangGraph interrupt + PostgreSQL 체크포인트).	28
4.4.4. 관제 및 알림 시스템	28
4.5. 전체 파이프라인 End-to-End 검증	37
5. 연구 결과 분석 및 평가	37
5.1. 시나리오 설계 개요	37

5.2.	시나리오별 실행 결과	38
5.3.	시스템 성능	38
5.3.1.	파이프라인 전체 성능	38
5.3.2.	비용	43
5.3.3.	실행 소요 시간	47
5.4.	시스템 성능 통계적 분석 (Clopper-Pearson 신뢰구간)	50
5.5.	버그 발견 및 수정 이력	51
6.	결론 및 향후 연구 방향	54
7.	구성원별 역할 및 개발 일정	56
8.	참고 문헌	57
8.1.	참고 문헌 목록	57

1. 요약

클라우드의 유연한 자원 확장은 비용 이상으로 이어질 수 있으며, 비용 낭비 및 비용 공격과 같은 비정상적인 비용 발생을 적시에 탐지하고 대응하는 것이 중요하다. 본 연구에서는 LangGraph 기반 멀티에이전트 파이프라인을 구축하여 비용 이상 탐지부터 분류·판단·조치·검증까지의 과정을 자동화하였다. 또한 실제 AWS 리소스와 트래픽을 활용하여 시스템의 동작과 성능을 검증하였다. 그 결과, 주요 시나리오에서 평균 탐지 정확도 86.7%(표본 수 가중평균), Recall 85.7%(95% CI [69.7%, 95.2%])를 확인하였고, 전 단계를 곱한 전체 파이프라인 정확도는 평균 79.7%로 나타났다. 또한 EC2 오버프로비저닝 시나리오만을 운영규모가 큰 조직의 운영 규모로 환산했을 때 30일 기준 약 \$763의 비용 절감 효과를 확인하였다.

2. 서론

2.1. 연구 배경

클라우드 컴퓨팅은 초기 투자 비용 없이 필요한 만큼만 자원을 사용할 수 있다는 장점으로 인해 기업의 IT 인프라 운영 방식을 근본적으로 바꾸어 왔으며, 그 사용 범위는 대기업뿐 아니라 개인 개발자와 소규모 팀 단위로까지 빠르게 확산되고 있다. 사용 주체가 늘어나고 의존도가 커질수록 클라우드 비용은 조직 운영에서 더 이상 무시할 수 없는 주요 운영 비용으로 자리 잡았다.

클라우드 환경에서는 리소스를 자유롭게 생성·확장할 수 있는 유연성이 오히려 비용 관리의 난제로 이어진다. 온프레미스 환경에서는 물리 서버 구매 및 증설에 결재와 리드타임이 필요해 비용이 급격히 늘어나는 일이 구조적으로 어려웠지만, 클라우드에서는 API 호출 한 번, 혹은 오토스케일링 정책 하나만으로 수십~수백 대의 인스턴스가 순식간에 기동될 수 있다. 개발/테스트 후 반납되지 않은 EC2 인스턴스, 재시도 로직의 오작동으로 폭증한 Lambda 호출, 트래픽 급증이나 악의적 요청(EDoS)으로 인한 과금 폭탄 등은 사람이 지속적으로 모니터링하지 않는 경우 사후 월말 청구서를 받은 이후에야 인지될 가능성이 높다.

클라우드 환경에서 발생하는 비용 낭비의 규모는 실제 통계에서도 확인할 수 있다. Flexera의 2026년 State of the Cloud Report에 따르면 IaaS/PaaS 지출 중 29%가 실질적인 가치를 만들어내지 못하는 낭비성 지출로 조사되었다.[1] 이는 클라우드 환경

에서 불필요한 자원 사용으로 인한 비용 낭비가 여전히 중요한 문제임을 보여준다. 또한 클라우드 비용 문제는 단순한 자원 낭비뿐만 아니라 EDoS와 같이 공격자가 의도적으로 비용을 발생시키는 경우에도 발생할 수 있다. 따라서 효과적인 클라우드 비용 관리를 위해서는 자원 사용의 비효율뿐만 아니라 비정상적인 비용 발생을 유발하는 보안 위협까지 함께 고려할 필요가 있다.

이러한 비용 관리의 필요성은 실제 산업 현장에서 FinOps(클라우드 재무 운영)라는 전문 분야가 별도로 자리 잡을 만큼 중요한 과제로 대두되고 있으며, 조직의 84%가 클라우드 지출 관리에 어려움을 겪고 있다고 응답한 조사 결과가 이를 뒷받침한다[2]. 특히 전담 FinOps 인력을 별도로 둘 여력이 없는 중소기업이나 스타트업의 경우, 비용 이상을 사람이 상시 모니터링하며 대응하는 것 자체가 현실적으로 어렵다.

본 연구에서는 전담 인력이 없는 환경에서도 클라우드 비용 이상을 탐지하고, 원인을 분석하여 적절한 조치를 수행한 후 그 결과를 검증할 수 있는 자동화된 비용 대응 파이프라인을 구현한다. 이를 통해 중소 규모 조직에서도 클라우드 비용 이상에 신속하게 대응할 수 있는 관리 체계를 제시하고자 한다.

2.2. 기존 문제점

2.2.1. 수동 모니터링 대응의 한계

AWS Cost Explorer/Budgets 등 기본 제공 도구는 일/월 단위 비용 집계와 사전에 설정한 예산 초과 알림 정도만 제공한다[9]. 즉 '얼마가 더 나왔는지'는 사후에 알 수 있지만, '무엇이 원인이고 어떤 리소스를 어떻게 조치해야 하는지'는 결국 담당자가 로그와 지표를 직접 뒤져 판단해야 한다. 이 구조에서는 탐지-원인 파악-조치까지 걸리는 시간이 담당자의 가용 시간에 종속되며, 야간·주말 등 관제 공백 시간대에 발생하는 이상 징후에는 사실상 무방비 상태가 된다.

특히 소규모 팀에서는 전담 FinOps 담당자를 따로 두기 어려워, 비용 이상 징후 확인이 후순위로 밀리는 경우가 많다. 담당자가 매일 대시보드를 확인한다고 가정하더라도, 확인 시점과 실제 이상 발생 시점 사이의 지연동안 비용은 계속 누적되며, 조치 역시 콘솔에서 인스턴스를 하나씩 찾아 수동으로 중지·재조정해야 하므로 조치 자체에도 적지 않은 시간이 소요된다. 이러한 수작업 중심 대응은 규모가 커질수록(관리 대상 리소스 수가 늘어날수록) 선형적으로 부담이 커진다는 근본적인 확장성 한계를 가진다.

2.2.2. 기존 FinOps 도구들의 한계 및 차별점

AWS Cost Anomaly Detection, CloudHealth, Vantage 등 상용/내장 FinOps 도구들은 비용 시계열 자체의 통계적 이상을 탐지하고 이를 알림 형태로 제공하는 데 초점을 둔다[10]. 실제 리소스의 중지·재조정이나 트래픽 제한과 같은 실행 단계는 별도의 운영 절차나 담당자의 개입이 필요한 경우가 많으며, 조치에 따른 서비스 영향 평가와 실패 시 롤백까지를 하나의 자동화된 흐름으로 통합하는 데에는 한계가 있다. 한편 AWS Auto Scaling과 같이 자원 조치 자체를 자동화하는 도구도 있으나, 이는 트래픽 증가가 정상적인 부하인지 비용 이상 또는 공격에 의한 것인지를 판단하는 기능과는 목적이 다르다. 특히 EDoS와 같이 정상적인 트래픽 증가로 위장한 공격이 발생하는 경우, Auto Scaling은 증가한 부하에 따라 자원을 확장할 수 있어 공격에 따른 비용 증가를 억제하기 어렵다는 한계가 있다.

본 프로젝트는 다음 세 가지 측면에서 기존 도구들과 차별화된다. 첫째, 단순한 비용 이상 여부만을 판단하는 것이 아니라, 탐지된 이상 징후를 리소스 및 상황에 따라 cost_inefficiency, cost_spike, risk_security로 세분화해서 분류하고, 각 유형에 적합한 대응 방안을 결정한다. 둘째, LangGraph 기반 멀티에이전트 구조를 통해 이상 유형에 따른 의사결정부터 조치 실행, 실행 결과에 대한 QA, 필요 시 롤백 및 결과 로깅까지를 하나의 자동화된 파이프라인으로 구성하였다. 이를 통해 기존의 탐지·알림 중심 접근에서 나아가 실제 클라우드 리소스에 대한 조치 실행까지 자동화 범위를 확장하였다. 셋째, 위험도가 높은 조치에는 관리자 승인 게이트를 적용하여 Human-in-the-loop 구조를 구현하였다. 또한 관리자가 비용 절감과 서비스 가용성 사이에 우선순위를 조정할 수 있도록 하고, 설정된 우선순위를 LLM의 의사결정 기준에 반영함으로써 자율성과 운영 안정성 사이의 균형을 고려한 자동화 구조를 설계하였다. (4.4.2절 참고). 본 절의 비교는 산학협력 멘토의 자문 의견을 반영하여 중간보고서 이후 추가되었다.

2.3. 연구 목표

본 연구의 목표는 클라우드 비용 이상을 실시간으로 탐지하고, 사람의 개입을 최소화하면서도 통제 가능한 방식으로 자동 대응하는 멀티에이전트 파이프라인을 구축하는 것이다. 이를 위해 다음과 같은 네 가지 목표를 설정하였다.

- 다양한 비용 이상 유형 탐지

좀비(유휴) 리소스, 오버프로비저닝, EDoS, Lambda 재시도 폭증, S3 대량 다운

로드 등 5가지 이상 유형을 EC2, Lambda, S3, Auto Scaling, RDS 등 실제 AWS 리소스에 걸쳐 탐지할 수 있는 체계를 구축한다.

- 규칙·LLM 기반 이상 판단

탐지된 이상의 분류와 대응 방안 결정에 규칙 기반 판단과 LLM 기반 판단을 함께 활용하여, 명확한 패턴은 신속하게 처리하고 모호한 패턴은 유연하게 대응하도록 구성하였다.

- 위험도 기반 승인 제어

위험도가 중간(MED) 이상인 조치는 관리자의 최종 승인을 거치는 승인 게이트를 적용하여 자동화에 따른 오작동 위험을 통제하였다. LangGraph의 interrupt()와 PostgreSQL 기반 checkpointer를 활용하여 승인 대기 상태를 유지하고, 승인 또는 거부 이후 중단된 지점부터 파이프라인을 재개하도록 구현하였다.

- 조치 결과 검증 및 롤백

조치 이후 실제 지표를 재조회하여 결과를 검증하고, 검증에 실패할 경우 이전 상태로 자동 롤백하도록 구성하여 조치의 신뢰성을 확보하였다.

이러한 기능은 LangGraph 기반의 이상 탐지(Detection) → 분류(Classification) → 액션 결정(Decision) → 액션 실행(Action) → 품질 검증(QA) → 로깅(Logging)의 6단계 멀티 에이전트 파이프라인으로 통합하고, 승인 대기 및 QA 실패에 따른 롤백 및 재시도 등의 조건부 분기를 그래프 수준에서 처리한다.

또한 AWS API와 CloudWatch를 연동하여 실제 리소스 상태 및 지표를 수집하고 Stop, Resize, Throttle, Block, Scale In 등의 조치를 수행하도록 구현한다. 특히 실제 AWS 리소스와 트래픽을 대상으로 전체 파이프라인의 동작을 검증함으로써, 클라우드 비용 이상에 실질적으로 대응할 수 있는 자율적이고 통제 가능한 비용 관리 체계를 구축하고자 한다.

2.4. 기대 효과

본 시스템은 비용 이상을 조기에 탐지하여 불필요한 지출이 누적되기 전에 대응하고, 이상 발생부터 조치 완료까지의 대응 시간을 단축할 수 있을 것으로 기대한다.

특히 담당자가 자리를 비우는 야간·주말 등 관제 공백 시간에도 자동으로 대응함으로써, 대응 지연에 따른 비용 피해의 확산을 시간대와 관계없이 억제할 수 있다. 또한 판단·조치·검증 이력을 PostgreSQL Audit Log와 Grafana 대시보드에 기록하여, 사후 감사와 Rule Book 고도화에 활용할 수 있는 데이터 기반을 확보한다.

승인 게이트를 통해 고위험 액션에만 사람의 개입을 요구함으로써 저위험 조치는 자동으로 처리하고, 관리자는 판단이 필요한 사안에 집중할 수 있도록 하여 운영 부담을 줄일 수 있다. 아울러 반복적으로 동일한 결정을 내리는 LLM 판단 패턴을 Rule Book의 규칙으로 승격함으로써, 이후 동일한 상황에서는 LLM 호출 없이 규칙 기반으로 처리할 수 있어 AI 호출 비용을 지속적으로 절감할 수 있다. 또한 리소스 유형·지표·조치 방식을 스키마 단위로 분리하여, 새로운 클라우드 자원이나 이상 유형이 추가되더라도 기존 파이프라인의 구조를 유지하면서 기능을 확장할 수 있도록 하였다.

관리자는 시스템의 자동화 수준을 일괄적으로 설정하는 대신, 대시보드의 우선순위 설정을 통해 서비스 가용성과 비용 절감 중 어느 측면에 더 높은 가중치를 부여할지 조정할 수 있다. 이를 통해 조직의 운영 환경과 요구사항에 따라 자동화의 판단 기준을 조정하고, 자동화 도입 과정에서 발생할 수 있는 운영상의 우려를 완화할 수 있을 것으로 기대한다.

3. 연구 배경

3.1. 배경 지식

3.1.1. 클라우드 비용 이상

본 연구에서 비용 이상은 클라우드 리소스의 실제 사용 패턴이나 비용 발생 양상이 정상적으로 기대되는 범위 또는 과거 이력에서 벗어나, 불필요한 지출이나 예상치 못한 지출 증가를 유발하는 상태를 의미한다.

본 시스템이 다루는 이상 유형은 크게 세 가지로 정의된다.

anomaly_type	정의	예시 시나리오	긴급도
cost_inefficiency	리소스가 정상적으로 동작은 하지만 낭비가 발생하는 상태(зом비 리소스, 오버프로비저닝)	장기간 저사용 EC2 인스턴스 방치	낮음
cost_spike	트래픽·호출 횟수 급증으로 단기간 비용이 튀는 상태	Lambda 재시도 폭증, 호출량 급증	높음

risk_security	악의적이거나 비정상적인 접근으로 인한 위험	EDoS, DDoS, 대량 다운로드	매우 높음
---------------	-------------------------	---------------------	-------

<표 1. 클라우드 비용 이상 유형 정의>

탐지 자체는 Z-score 기반 단변량 임계값 판정과 Isolation Forest 기반 다변량 이상 스코어링을 OR 게이트로 결합하여, 둘 중 하나라도 이상으로 판단하면 해당 리소스를 이상으로 확정하는 방식을 기본 판정 근거로 삼는다. 여기에 EC2 유휴 리소스처럼 통계적 변화가 크지 않더라도 절대적인 수치 기준에서 명확한 이상으로 판단할 수 있는 경우를 탐지하기 위해 절대임계값 판정을 보조 경로로 추가하였다.(4.2.1 절 참고) 이처럼 서로 다른 판정 경로를 병렬적으로 결합함으로써 단일 알고리즘의 맹점을 상호 보완하도록 설계하였다.

각 이상 유형은 원인에 따라 구체적인 시나리오로 세분화되며, 판정 기준은 다음과 같다.

시나리오	정의	문제	판정 기준
좀비(유휴) 리소스	실행 중이지만 실질적인 사용이 거의 없는 리소스	실질적인 작업을 수행하지 않으면서도 지속적으로 비용이 발생하여 불필요한 지출로 이어짐	피크 CPU 사용률 5% 이하 기준 (AWS Compute Optimizer의 idle 정의)
오버프로비저닝	실제 사용량이나 필요 수준에 비해 과도하게 큰 사양이 할당된 리소스	실제로 활용되지 않는 용량에 대해서도 비용이 발생하여 지속적인 비용 낭비로 이어짐	저활용 기준 (10%) + 여유 마진 (AWS Trusted Advisor 정의)
Lambda 스로틀 재시도 폭증	예약된 동시 실행 한도를 초과하여 함수 호출이 스로틀링되고 비동기 재시도가 반복되는 상태	반복되는 재시도로 인해 불필요한 비용이 지속적으로 발생함	편차기반(z-score) + 모델기반(IsolationForest) OR 게이트 탐지
S3 대량 다운로드	짧은 시간 동안 비정상적으로 많은 다운로드 요청 또는 데이터 전송이 발생하는 현상	데이터 전송량에 따라 비용이 발생하는데, 악의적인 대량 다운로드는 비용 증가를 유발할 수 있음	편차기반(z-score) + 모델기반(IsolationForest) OR 게이트 탐지
AutoScaling EDoS	정상적인 트래픽 증가로 위장한 대량 요청을 통해 Auto Scaling을 유발하고 클라우드 비용을 증가시키는 공격	공격과 정상적인 트래픽 증가를 구분하지 못하면 자동 확장으로 인해 불필요한 리소스가 추가로 기동되어	편차기반(z-score) + 모델기반(IsolationForest) OR 게이트 탐지

		비용이 증가할 수 있음	
--	--	--------------	--

<표 2. 이상 유형에 따른 시나리오 정의>

3.1.2. 멀티에이전트 파이프라인

멀티에이전트 파이프라인은 하나의 인공지능 모델이 모든 작업을 수행하는 대신, 문제를 여러 개의 독립적인 기능 단위로 분해하고 각 단위가 서로 협력하여 전체 작업을 수행하는 구조를 의미한다. 각 에이전트는 특정 역할과 책임을 담당하며, 에이전트 간의 정보 전달과 실행 순서를 정의하는 워크플로우를 통해 복합적인 문제를 단계적으로 처리할 수 있다. 이러한 구조는 하나의 판단 로직에 모든 기능이 집중되는 것을 방지하고, 각 기능을 독립적으로 구성함으로써 시스템의 확장성과 유지보수성을 높일 수 있다는 장점이 있다.

클라우드 비용 이상탐지 및 자율 복구 문제는 단순히 비용의 이상 여부를 판단하는 것에서 끝나지 않고, 이상 징후의 분류, 원인 및 상황 분석, 적절한 대응 조치의 결정, 실제 리소스에 대한 조치 실행, 실행 결과의 검증과 실패 시 복구 등 서로 다른 성격의 작업을 연속적으로 수행해야 한다. 따라서 이러한 작업을 역할별 에이전트로 분리하고 각 단계의 결과를 다음 단계로 전달하는 멀티 에이전트 구조를 적용하면, 탐지부터 대응 및 검증까지의 복합적인 의사결정 과정을 단계적으로 구성할 수 있다.

본 시스템에서는 탐지(Detection), 분류(Classification), 결정(Decision), 실행(Action), 검증(QA), 기록(logging)의 기능을 각각 독립적인 노드로 구성하고, 공통 state를 통해 각 단계에서 생성된 정보를 전달한다. 탐지 단계에서 생성된 이상 여부와 이상 점수는 이후 분류 및 의사결정 단계의 입력으로 활용되며, 결정 단계에서 선택된 조치는 실행 단계로 전달된다. 실행 이후에는 QA 단계를 통해 조치 결과를 검증하고, 검증 결과에 따라 추가 조치 또는 롤백 여부를 결정하도록 구성하였다.

또한 각 노드 간의 연결을 그래프 구조로 정의하고 조건부 라우팅을 적용함으로써, 처리 결과에 따라 서로 다른 경로로 전환되는 동적 제어 흐름을 구성할 수 있다. 예를 들어 이상 징후가 탐지되지 않은 경우 후속 처리 없이 종료할 수 있으며, 조치 이후 검증 결과가 기준을 충족하지 못한 경우 복구 절차를 수행할 수 있도록 구성할 수 있다. 이를 통해 본 시스템은 단순히 이상을 탐지하고 알림을 전달하는 것을 넘어, 탐지 결과를 기반으로 판단하고 조치를 수행한 후 그 결과를 다시 검증하는 폐

쇄형(Closed-loop) 처리 구조를 구현한다.

이와 같은 멀티 에이전트 구조는 각 단계의 책임을 분리함으로써 특정 기능의 변경이 다른 기능에 미치는 영향을 최소화하고, 새로운 탐지 기법이나 대응 조치를 추가할 때 기존 파이프라인의 구조를 크게 변경하지 않고 확장할 수 있다는 장점이 있다.

3.1.3. 클라우드

클라우드 컴퓨팅은 서버·스토리지·네트워크 등의 IT 자원을 필요한 만큼 빌려 쓰고, 사용량 또는 선택한 요금제에 따라 과금되는 컴퓨팅 환경이다. 흔히 탄력성과 사용량 기반 과금이 클라우드의 핵심 특성으로 꼽히는데, 이는 트래픽 변화에 따라 자원을 신속하게 확장하거나 축소할 수 있다는 장점을 제공한다. 그러나 이러한 유연성은 자원의 확장이 비용 증가로 직접 이어질 수 있다는 측면에서, 본 연구가 다루는 비용 이상탐지 문제의 근본 배경이 된다.

본 시스템이 다루는 AWS 리소스 유형(EC2, Lambda, S3, AutoScaling)은 각각 서로 다른 과금 구조와 사용 특성을 갖는다: EC2는 주로 인스턴스의 사용 시간과 인스턴스 유형에 따라 비용이 발생하며, Lambda는 호출 횟수와 실행 시간 및 메모리 사용량에 따라 과금된다. S3는 저장 용량과 요청 및 데이터 전송량 등에 따라 비용이 발생하며, Auto Scaling은 설정된 정책에 따라 EC2 등의 인스턴스 수를 자동으로 증감 시키므로 실제 기동된 인스턴스의 수와 사용 시간에 따라 비용이 증가할 수 있다. 이러한 차이로 인해 비용 이상의 발생 양상과 대응 방식 역시 리소스 유형에 따라 달라질 수 있으며, 본 연구에서는 이를 리소스 및 시나리오별 조치 매핑으로 구체화하였다.

3.1.4. FinOps

FinOps는 클라우드 비용을 엔지니어링, 재무, 비즈니스 조직이 함께 책임지고 최적화하는 운영 문화이자 방법론이다[10]. 일반적으로 FinOps의 활동은 Inform(가시화) → Optimize(최적화) → Operate(운영)의 단계로 설명되며, 이를 통해 조직의 클라우드 비용 관리 수준을 지속적으로 개선하는 것을 목표로 한다. 본 시스템은 이 중 Optimize~Operate 단계에 해당하는 비용 관리 자동화에 초점을 두며, 비용 이상을 탐지하고 원인을 분석하여 실제 조치까지 수행하는 것을 목표로 한다.

FinOps Foundation은 이 방법론의 핵심 원칙 중 하나로, 팀이 비즈니스 가치를 위

해 빠르게 움직이면서도 비용을 인지하고 관리해야 한다는 점을 강조한다[10]. 본 시스템의 Grafana 대시보드는 '시스템 미적용 시 예상 비용', '시스템 적용 시 비용', '절감액' 등의 정보를 실시간으로 제공하도록 설계하였으며, 이를 통해 비용 변화와 자동화에 따른 절감 효과를 가시적으로 확인할 수 있도록 하였다.

3.1.5. Isolation Forest

Isolation Forest는 Liu 등이 제안한 트리 기반 비지도 이상탐지 알고리즘으로[3], 데이터를 무작위로 분할해 나갈 때 정상 데이터는 다른 데이터들과 유사해 상대적으로 많은 분할을 거쳐야 고립되는 반면, 이상치는 적은 분할만으로도 쉽게 고립된다는 직관에 기반한다. 각 데이터 포인트 x 에 대해 여러 개의 무작위 이진 트리(iTree)를 만들어 평균 경로 길이 $E(h(x))$ 를 구하고, 이를 정규화해 이상 스코어 $s(x, n) = 2^{(-E(h(x))/c(n))}$ 로 산출한다. 여기서 $c(n)$ 은 n 개의 데이터를 이진 탐색트리(BST)로 분할했을 때 기대되는 평균 경로 길이를 정규화하기 위한 상수이다. 스코어가 1에 가까울수록 이상, 0에 가까울수록 정상으로 해석하며, 0.5에 가까운 경우는 뚜렷한 이상치로도 정상으로도 보기 어려운 애매한 상태를 의미한다 [3].

Isolation Forest는 거리나 밀도 기반 방법(k-NN, LOF 등)과 달리 이상치를 '거리가 먼 점'이 아니라 '고립시키기 쉬운 점'으로 정의하기 때문에 계산 복잡도가 선형에 가깝고, 레이블이 없는 상황에서도 여러 지표(CPU 사용률, 네트워크 입출력, 비용 등)를 동시에 고려하는 다변량 판정이 가능하다는 점에서 단일 지표만 보는 Z-score의 한계를 보완한다. 본 시스템에서의 구체적인 파라미터 선정과 학습 방식은 4.3.2~4.3.3절에서 자세히 다룬다.

3.1.6. Z-Score

Z-score는 관측값이 해당 지표의 평균에서 표준편차 몇 배만큼 떨어져 있는지를 나타내는 표준화 통계량으로, $Z = (x - \mu) / \sigma$ 로 정의된다(μ : 평균, σ : 표준편차). 정규분포를 가정할 때 $|Z| > 3$ 인 값은 이론적으로 전체의 약 0.3% 이내에 속하는 극단값이며, 본 시스템은 비용, 네트워크 입력, 호출 횟수 등 개별 지표에 대해 $k=3.0$ 을 초기 기본값으로 두고, 4.3.4절의 그리드 서치 기준 최적값으로 $k=2.75$ 를 채택하였다. 순간적인 노이즈로 인한 오탐(flapping)을 막기 위해, 최근 3개 연속 시점의 $|Z\text{-score}|$ 가 모두 이 임계값을 넘을 때만 해당 지표를 '트리거된 지표(triggered_metrics)'로 기록한다.

계산이 단순하고 해석이 직관적이라는 장점이 있는 반면, 지표를 하나씩만 보기

때문에 여러 지표가 동시에 미묘하게 변하는 복합 이상 패턴은 놓치기 쉽다는 한계가 있다. 이런 한계는 Isolation Forest와 OR 게이트로 상호 보완한다.

3.1.7. 이상탐지 검증 지표: Clopper-Pearson 신뢰구간

일반적으로 통계적 추정에서 신뢰구간은 모수(참값)가 특정 확률(신뢰수준)로 포함될 것으로 기대되는 구간을 의미한다. 표본 수가 충분히 크면 정규분포 근사(Wald 구간)를 써도 무방하지만, 본 연구처럼 실 AWS 리소스를 대상으로 한 반복 시행 횟수가 $n=13$ 수준으로 작을 때는 정규근사가 실제 분포를 왜곡할 수 있다.

이를 보완하기 위해 이항분포의 정확 신뢰구간인 Clopper-Pearson 구간을 함께 산출하여[5], 예를 들어 '5회 시행 중 5회 성공(재현율 100%)'이라는 결과가 실제로는 95% 신뢰수준에서 어느 범위의 참값을 가질 수 있는지까지 함께 보고한다. 구체적인 계산 방식과 산출 결과는 4.3절 및 각주를 참고한다.¹

3.1.8. LangGraph

LangGraph는 LLM 기반 애플리케이션을 방향성 그래프(노드=작업 단위, 엣지=제어 흐름)로 표현하는 오케스트레이션 프레임워크이다[6]. 본 시스템이 사용하는 핵심 개념은 세 가지다.

- StateGraph: 모든 노드가 하나의 공유 State(TypedDict)를 입력받아 갱신된 State를 반환하는 방식으로 연결된 그래프. 조건부 엣지(add_conditional_edges)를 사용하면 이전 노드의 출력값에 따라 다음에 실행할 노드를 동적으로 결정할 수 있다.
- 체크포인트(Checkpointer): 그래프 실행 중 각 노드가 끝날 때마다의 State 스냅샷을 외부 저장소(본 시스템은 PostgreSQL, PostgresSaver)에 영속화하는 컴포넌트[6]. 저장 단위는 thread_id로 구분되며, 같은 thread_id로 다시 invoke()하면 마지막으로 저장된 지점부터 실행이 이어진다.
- interrupt(): 노드 함수 내부에서 호출하면 그래프 실행을 그 자리에서 물리적

¹ Clopper-Pearson 구간은 이항분포의 정확(exact) 신뢰구간으로, 정규근사에 의존하지 않아 표본 수가 적을 때도 보수적으로 타당한 구간을 제공한다. 성공 횟수 x , 시행 횟수 n , 신뢰수준 $1-\alpha$ 일 때 하한은 $\text{Beta}(\alpha/2; x, n-x+1)$ 의 분위수, 상한은 $\text{Beta}(1-\alpha/2; x+1, n-x)$ 의 분위수로 계산되며($x=0$ 이면 하한 0, $x=n$ 이면 상한 1), 본 연구에서는 scipy.stats.beta 기반으로 구현하였다(playground/measure_classification_accuracy.py).

으로 멈추고 제어권을 호출자에게 돌려준다[7]. 이후 Command(resume=...)로 재개하면, interrupt()를 호출했던 지점부터 그 반환값을 받아 나머지 로직을 이어서 실행한다. 단, 재개 시 해당 노드 함수 자체는 처음부터 다시 실행되므로, interrupt() 호출 이전에 부수효과(알림 전송 등)가 있으면 재개 시 중복 실행될 수 있다는 점에 유의해야 한다(4.4.3/4.4.4절 참고).

이 세 요소가 결합되면 '사람이 개입하기 전까지 프로세스 자체가 메모리에서 사라져도(서버 재시작 포함) 나중에 정확히 그 지점부터 재개할 수 있는' 워크플로우가 가능해진다. 본 시스템의 승인 게이트(4.4.3절)가 이 조합을 활용한 사례이다.

3.1.9. Human-in-the-loop (HITL)와 승인 기반 자동화

Human-in-the-loop(HITL)은 자동화된 시스템의 의사결정 과정에 사람의 판단과 개입을 개입을 결합하는 설계 패러다임을 의미한다. AI 시스템의 판단을 항상 사람에게 의존하는 방식과 반대로 모든 의사결정을 완전히 자동화하는 방식 사이에서, 시스템이 반복적이고 위험도가 낮은 작업은 자동으로 처리하면서 불확실하거나 영향도가 높은 상황에서는 사람에게 판단 권한을 부여할 수 있다는 점에서 활용된다. 특히 클라우드 운영 환경에서는 자동화된 조치가 비용 절감 효과를 가져오는 동시에 서비스 가용성이나 안정성에 영향을 미칠 수 있으므로, 조치의 위험도에 따라 사람의 개입 수준을 조절하는 방식이 중요하다[13].

본 시스템에서는 이러한 HITL 개념을 조치의 위험도(risk_level)에 따른 승인 기반 자동화 방식으로 적용하였다. 이상 유형에 따라 기본 위험도를 설정하고, 선택된 조치가 더 높은 위험도를 요구하는 경우 해당 위험도를 상향하여 최종 위험도를 결정한다. cost_inefficiency는 LOW, cost_spike는 MED, risk_security는 HIGH를 기본 위험도로 설정하며, Stop+Schedule 및 Resize와 같은 조치는 최소 MED, Block은 HIGH 이상의 위험도를 요구하도록 구상하였다. 이를 통해 단순히 이상 유형만으로 자동 실행 여부를 결정하지 않고, 실제 수행할 조치의 영향 수준까지 고려하여 승인 필요 여부를 판단한다.

최종 위험도가 LOW인 경우에는 별도의 승인 절차 없이 조치를 자동으로 실행하며, MED 또는 HIGH인 경우에는 관리자 승인 게이트를 거치도록 구성하였다. 즉, 반복적이고 상대적으로 영향이 낮은 작업은 시스템이 자율적으로 처리하는 반면, 서비스 중단이나 보안과 같이 영향도가 높은 조치에는 관리자의 판단을 개입시킨다. 이를 통해 자동화에 따른 운영 효율성을 확보하면서도 고위험 의사결정에 대한 통제권을 관리자에게 유지하는 구조를 구현하였다.

또한 승인 기반 자동화에서는 관리자의 승인 시점이 시스템의 자동 실행 시점과 일치하지 않을 수 있다는 비동기적 특성을 고려해야 한다. 본 시스템은 승인 요청이 발생한 시점에서 파이프라인을 일시 중단하고 실행 상태를 보존한 뒤, 관리자의 승인 또는 거부가 전달되었을 때 동일한 실행 흐름을 재개하는 방식으로 이를 처리한다. 따라서 승인 여부를 확인하기 위해 별도의 반복적인 대기나 폴링을 수행하지 않고도, 관리자의 판단 이후 기존 파이프라인을 이어서 실행할 수 있다.

이와 같은 구조를 통해 본 시스템은 모든 작업을 무조건 자동화하는 방식에서 벗어나, 조치의 위험도에 따라 자동화 수준을 차등 적용하는 자율 운영 방식을 지향한다. 이는 비용 절감 효과가 큰 저위험 작업은 신속하게 자동 처리하면서도, 서비스 안정성이나 보안에 영향을 줄 수 있는 고위험 작업에는 사람의 최종 통제권을 유지한다는 점에서 클라우드 비용 이상탐지 및 자율 복구 시스템에 적합한 접근이다.

3.1.10.WAF와 Rate-based Rule

AWS WAF(Web Application Firewall)는 웹 애플리케이션으로 유입되는 요청을 규칙(Rule)에 따라 검사하고 허용 또는 차단하는 관리형 웹 방화벽이다[12]. 이 중 Rate-based Rule 은 5 분 동안 동일한 집계 기준(IP 등)에서 발생하는 요청 수를 측정하고, 설정된 임계값을 초과하는 요청에 대해 해당 출처의 트래픽을 자동으로 차단하는 기능이다. 따라서 짧은 시간 동안 대량의 요청이 발생하는 DDoS/EDoS 와 같은 비정상 트래픽을 완화하는 데 활용할 수 있다.

본 시스템에서는 클라우드 비용 이상이 대량의 트래픽이나 비정상적인 요청 증가로 인해 발생하는 경우를 고려하여, 탐지된 이상 상황에 대한 트래픽 제어 수단으로 WAF 를 활용한다. 특히 AWS Security Group 은 인바운드 트래픽에 대해 허용(allow) 규칙을 기반으로 접근을 제어하며, 특정 트래픽을 직접 거부(deny)하는 방식의 규칙을 제공하지 않는다. 따라서 특정 출처의 트래픽을 차단하기 위해서는 WAF 의 차단 규칙이나 Network ACL(NACL)의 deny 규칙과 같은 별도의 제어 수단이 필요하다.

본 시스템은 이 중 WAF 를 활용하여 Rate-based Rule 과 IPSet 기반의 IP 블랙리스트를 트래픽 차단 조치로 사용한다. Rate-based Rule 은 일정 시간 동안 과도하게 증가한 요청을 자동으로 제한하는 데 활용하고, IP 블랙리스트는 이상 트래픽의 출처로 판단된 특정 IP 에 대해 선택적으로 차단을 수행하는 방식이다.

이를 통해 비용 이상 탐지 결과를 단순한 경고에 그치지 않고 실제 인바운드 트래픽 제어 조치로 연결할 수 있도록 구성하였다.

3.1.11.SLA와 가용성 지표

SLA(Service Level Agreement, 서비스 수준 협약)는 서비스 제공자와 이용자 사이에 합의된 서비스의 성능 및 가용성 수준을 정량적인 지표로 정의한 것이다. 클라우드 환경에서는 서비스의 가용성, 응답 성능 및 장애 대응 수준 등을 기준으로 서비스 품질을 평가할 수 있으며, 자동화된 운영 시스템에서는 특정 조치가 서비스에 미치는 영향을 지속적으로 확인하는 것이 중요하다.

본 시스템에서는 이러한 SLA 개념을 자율 복구 조치의 안전성을 검증하기 위한 내부 기준으로 활용하였다. QA Agent 는 조치 실행 직후 메트릭(CPU 사용률 등), 비용 변화 및 액션 성공 여부를 확인하여 조치가 정상적으로 수행되었는지를 자동으로 검증한다. 구체적으로 CPU 사용률은 80% 이하를 허용 기준으로 설정하고, 비용은 조치 이전 대비 10% 이상 증가할 경우 검증 기준을 충족하지 못한 것으로 판단한다. 또한 액션이 정상적으로 완료되었는지를 확인하여 서비스 가용성 유지 여부를 함께 판단한다.

이러한 지표는 일반적인 서비스 제공 계약에서 사용하는 장기간의 SLA 지표와 동일한 의미의 계약상 SLA 가 아니라, 본 시스템에서 자동화된 조치의 안정성을 즉시 확인하기 위해 정의한 단기 검증 기준이다. 즉, 조치 실행 직후 QA 단계를 수행하여 CPU 사용률과 비용 변화 및 액션 결과를 확인하고, 기준을 충족하지 못한 경우 추가 조치 또는 롤백으로 이어질 수 있도록 구성하였다.

따라서 본 시스템에서 SLA 및 가용성 검증은 단순히 조치의 성공 여부만을 확인하는 것이 아니라, 자동화된 복구 조치가 비용 절감이라는 목적을 달성하면서도 서비스 운영에 부정적인 영향을 발생시키지 않았는지를 확인하는 안전장치의 역할을 한다. 이를 통해 자율 복구 과정에서 발생할 수 있는 잘못된 조치의 영향을 조기에 감지하고 후속 복구 절차로 연결할 수 있도록 하였다.

4. 연구 내용

4.1. 시스템 아키텍처 및 기술 스택

4.1.1. 전체 아키텍처 개요

본 시스템은 크게 (A) 지표·비용 수집, (B) 이상 탐지 및 자율 대응 파이프라인, (C) 관제·알림의 세 계층으로 구성된다.

먼저 지표·비용 수집 계층에서는 태그 기반 리소스를 선정한다. AWS 계정 내 모든 리소스를 무차별적으로 탐색하는 대신, Detection=true 태그가 지정된 리소스를 탐지 대상으로 선정한다. 선정된 리소스에 대해서는 Amazon CloudWatch의 운영 지표와 비용 관련 데이터를 수집하고, 이를 이상 탐지 및 자율 대응 파이프라인의 입력 데이터로 구성한다. 이후 각 대상 리소스에 대해 LangGraph 기반 파이프라인을 한 번 호출하여 탐지부터 후속 대응까지 하나의 실행 흐름으로 처리한다.

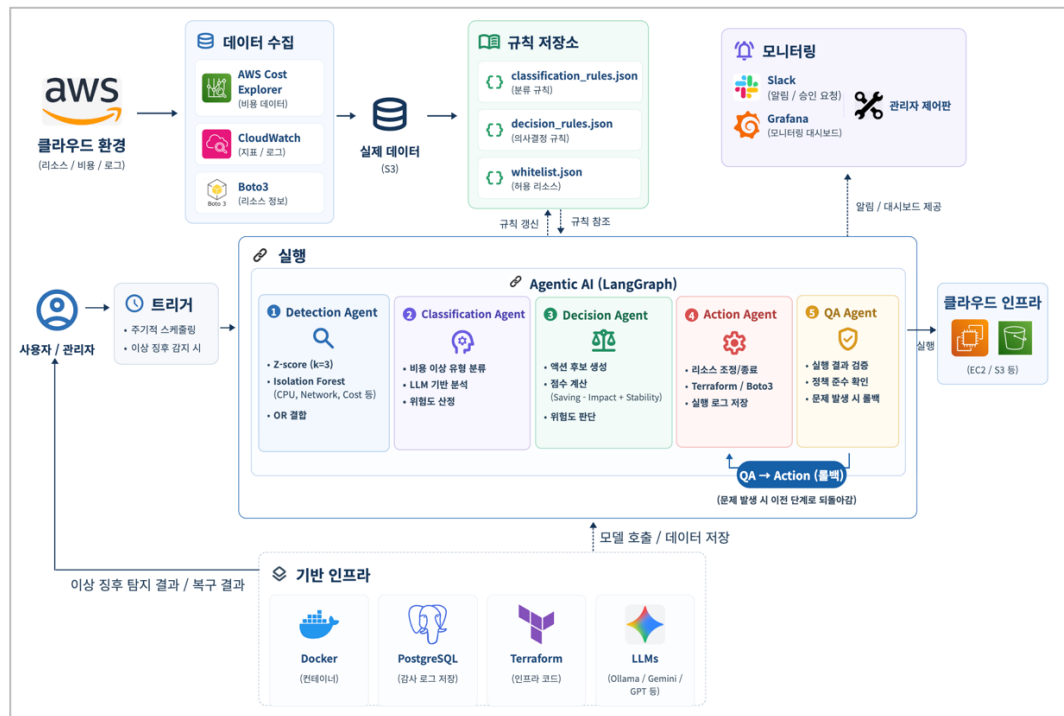
자율 대응 파이프라인은 Detection → Classification → Decision → Action → QA → Logging의 6 단계로 구성된다. 그래프의 진입점은 Detection이며, 탐지 결과에 따라 detection_router가 후속 실행 경로를 결정한다. 이상이 탐지된 경우 Classification 단계에서 이상 유형을 분류하고, 이후 Decision, Action, QA 단계를 거쳐 조치 결과를 검증한 후 Logging 단계에서 실행 이력을 기록한다. 반면 정상으로 판단된 경우에는 후속 대응 단계를 수행하지 않고 Logging 단계로 이동하여 실행을 종료한다. 이를 통해 정상 리소스에 대한 불필요한 LLM 호출과 후속 대응 작업을 방지한다.

MED 이상의 위험도가 부여된 조치는 Decision과 Action 사이에서 notify_gate와 approval_gate를 거치도록 구성하였다. notify_gate에서는 Slack을 통해 관리자에게 조치 내용을 통지하고, approval_gate에서는 관리자 승인 여부를 확인한다. 승인 대기 시에는 LangGraph의 Checkpointer를 통해 현재 실행 상태를 저장하며, 관리자 웹 인터페이스에서 승인 또는 거부가 이루어진 이후 저장된 상태를 기반으로 파이프라인을 재개한다. 이를 통해 장시간의 승인 대기가 발생하더라도 기존 실행 흐름과 상태를 유지할 수 있도록 구성하였다.

관제·알림 계층에서는 PostgreSQL을 실행 이력 및 감사 로그의 저장소로 활용하고, Grafana를 통해 탐지 및 조치 결과와 비용 절감 현황을 시각화한다.

또한 주요 이벤트와 승인 대기 및 조치 관련 상태를 Slack 으로 통지하여 관리자가 시스템의 실행 상황을 확인할 수 있도록 구성하였다.

관리자 제어 기능에서 사용하는 정책값은 파이프라인 워커 프로세스와 FastAPI 프로세스 사이에서 공유되어야 한다. 두 프로세스는 서로 독립적으로 실행되므로 메모리를 직접 공유하지 않고, config/decision_policy.json 을 공유 설정 저장소로 활용하여 정책값을 동기화한다. 이를 통해 가용성과 비용 절감 간 우선순위, LLM 비용 상한, 폴링 주기 및 리소스별 탐지 활성화 여부 등의 정책을 관리자 제어 화면에서 변경하고 파이프라인에 반영할 수 있도록 구성하였다. 또한 Rule Book 에서 JSON 기반 규칙 파일을 활용하는 방식과 동일한 파일 기반 설정 구조를 적용하여 시스템 전반의 설정 관리 방식을 일관되게 유지하였다.



<그림 1. 시스템 아키텍처>

4.1.2. 기술 스택 선정 근거

구분	기술/라이브러리	버전	선정 근거
오케스트레이션	LangGraph	1.1.10	롤백 사이클·승인 대기과 같은 복잡한 실행 흐름을 간결하게 구현할 수 있고, 순수 Python 대비 동일 기능 코드량을 약 68.8% 줄일 수 있어 선정하였다.[6][7]
LLM 추론	langchain-google-genai (Gemini)	4.2.2	무료 티어를 통한 대량 LLM 호출 비용 절감과 LangChain 생태계 기반의 파이프라인 통합 용이성

			을 고려하여 Gemini를 선정하였다 .
AWS 연동	boto3 / botocore	1.43.3	CloudWatch 지표 수집과 EC2, Lambda, S3, Auto Scaling 등의 리소스에 대한 실제 조치 및 WAFv2 Rate-based Rule 적용 등 AWS 서비스와의 연동을 구현하기 위해 AWS 공식 SDK를 사용하였다[8].
이상 탐지	scikit-learn (IsolationForest)	1.7.2	사전에 이상 여부에 대한 레이블을 확보하기 어려운 클라우드 운영 환경을 고려하여, 레이블이 없는 다변량 지표에서도 이상 패턴을 탐지할 수 있는 비지도 학습 기반 Isolation Forest를 적용하였다 [3][11].
해석가능성	SHAP	0.52.0	Isolation Forest의 이상 판정에 영향을 미친 지표별 기여도를 분석하여, 이상 탐지 결과에 대한 근거를 QA 과정 및 관리자 화면에서 확인하기 위해 선정 하였다[4].
Audit 저장소	PostgreSQL	16	새로운 리소스 유형 추가에 대한 확장성이 높고, JSONB를 활용하여 리소스별로 상이한 데이터를 유연하게 저장할 수 있기에 선정하였다 .
시각화	Grafana	11.4.0	PostgreSQL과의 네이티브 연동을 통해 별도의 백 엔드 구현이나 ETL 없이 운영 데이터를 효율적으로 시각화할 수 있고, 오픈소스로 비용 부담이 적어 선정하였다 .
관리자 웹 백엔드	FastAPI	0.115.6	Pydantic 기반 입력 검증, Depends()를 활용한 일관 된 인증 적용, lifespan을 통한 파이프라인 프로세스 관리, 자동 생성 OpenAPI 문서를 통한 협업 용이 성을 고려하여 FastAPI를 선정하였다.
관리자 웹 프론트	React (Vite)	^18.3.1	승인 대기 큐, Rule Book, 화이트리스트, 규칙 승격, 실패 이력 및 정책 설정 등의 관리 기능을 하나의 웹 인터페이스에서 제공하기 위해 선정하였다.
알림	Slack Incoming Webhook (requests)	2.33.1	파이프라인의 주요 실행 상태와 승인 대기, 조치 및 룰백 관련 이벤트를 관리자에게 실시간으로 전달하기 위해 선정하였다.

<표 3. 기술 스택 선정 근거>

4.2. 에이전트

4.2.1. Detection Agent

파이프라인의 진입점으로, 주기적으로 AWS 리소스의 운영 지표를 수집하여 이상 여부를 판단한다. Z-score를 활용한 단기 스파이크 탐지, Isolation Forest를 활용한 다변량 이상 탐지, EC2 저사용률 절대임계값을 병렬로 적용하고, 각 탐지 결과를 OR 양상블로 결합하여 이상 여부를 판단한다. 또한 여러 리소스 유형의 지

표를 통합된 feature 형태로 구성하여 하나의 Isolation Forest 모델에서 처리하도록 하였다. 운영 환경에서는 정상으로 판단된 데이터를 학습 버퍼에 축적하여 모델을 재학습할 수 있는 구조를 구성하였으나, 본 보고서의 성능 평가에서는 재현성을 위해 초기 학습 버퍼를 동결하여 사용하였다.

- **입력:** raw_metrics(지표별 30 개 슬라이딩 윈도우 리스트), resource_type
- **처리:** Z-score + IForest + 저활용 절대임계값 (OR 결합)
- **출력:** anomaly_flag, anomaly_score_zscore, anomaly_score_iforest, triggered_metrics

4.2.2. Classification Agent

탐지된 이상 신호를 바탕으로 anomaly_type ('cost_inefficiency', 'risk_security', 'cost_spike')을 분류한다. 먼저 Rule Book을 통해 명확하게 정의된 조건을 규칙 기반으로 매칭하고, 규칙만으로 판단하기 어려운 경우에만 LLM(Gemini)을 활용하여 분류한다. LLM을 통해 판단된 결과는 로그로 기록하며, 동일한 판단 패턴이 반복되고 QA를 통과한 경우에는 Rule Book에 등록할 후보로 관리하여 반복적인 LLM 추론을 규칙 기반 판단으로 전환할 수 있도록 구성하였다.

- **입력:** triggered_metrics, raw_metrics, resource_type
- **처리:** Rule Book(CLF-*) 우선 매칭 → 실패 시 LLM
- **출력:** anomaly_type, classification_reasoning, interim_action_taken

4.2.3. Decision Agent

분류된 anomaly_type과 현재 리소스 상태를 바탕으로 적용 가능한 조치를 검토하고 최종 액션을 하나 선택한다. 예상 절감률과 예상 절감액은 LLM이 임의로 추정하지 않고 raw_metrics의 비용 데이터를 기반으로 사전에 계산하여 제공하며, LLM은 각 조치의 비용 효과, 서비스 영향, 부작용 및 복구 가능성 등을 고려하여 최종 조치를 결정한다. 또한 관리자가 설정한 비용 절감과 서비스 가용성 간의 우선순위 정책을 프롬프트에 반영하여 동일한 이상 상황에서도 정책에 따라 다른 조치가 선택될 수 있도록 구성하였다. 최종 조치의 위험도를 LOW, MED, HIGH로 구분하고, MED 이상의 위험도를 가진 조치는 관리자 승인을 거치도록 설정한다.

- **입력:** anomaly_type, resource_type, raw_metrics(cost 시계열 포함)
- **처리:** Rule Book(DEC-*) 우선 매칭 → 실패 시 LLM 이 boto3 스펙 보고 액션 선택.
saving_rate/estimated_saving_usd 는 LLM 이 아니라 cost 시계열로 결정론적 계산
- **출력:** candidate_actions, selected_action, risk_level, requires_approval,

또한 Decision Agent의 LLM 프롬프트는 선택된 액션뿐 아니라 그 판단 로직을 if-else 한 줄 형태의 pseudo_code로 함께 출력하도록 요청하였다. 축적된 판단 로그를 분석한 결과 동일한 리소스 유형·이상 유형 조합에서 액션과 pseudo_code 가 모두 일관되게 반복되는 패턴을 다수 확인할 수 있었으며, 이러한 패턴은 매번 LLM을 호출하지 않고 규칙 기반 판단으로 전환할 수 있는 후보로 판단하였다. 이에 따라 반복 횟수와 일관성 비율이 기준을 만족하는 패턴을 자동으로 탐지해 규칙 승격 후보 대기열에 등록하는 기능을 구현하였으며, 오탐 패턴이 그대로 규칙으로 굳어지는 것을 방지하기 위해 승격은 자동 반영이 아니라 관리자 승인을 거치도록 설계하였다.

Decision Agent의 LLM 판단 로그(schema/logs/llm_decision_log.jsonl)를 리소스 타입과 이상 유형의 조합별로 묶어, 동일한 조합에서 선택된 액션과 pseudo_code 가 어느 정도 일관되게 반복되는지를 분석하였다. 각 조합은 최소 3회 이상 관측된 경우만 분석 대상에 포함하였다.

EC2/RDS + cost_inefficiency 조합은 액션 일치율이 각각 100%로 높게 나타났으나, pseudo_code 일관성은 67% 또는 33%에 그쳐 동일한 액션을 선택하더라도 세부 판단 조건은 완전히 일치하지 않는 것으로 나타났다. 따라서 현재 분석 결과만으로는 액션 선택의 반복성은 확인되지만, 승격 조건인 액션 일관성 80% 이상을 만족하더라도 pseudo_code 수준의 조건까지 일관되게 정형화되었다고 보기는 어렵다. 반면 Lambda + cost_spike 및 S3 + risk_security 조합은 액션 일치율 자체가 67%로 나타나 현재 기준의 승격 조건을 충족하지 못하였다. 이를 통해 반복적으로 동일한 판단과 액션이 나타나는 조합과 그렇지 않은 조합을 구분하고, 충분한 일관성이 확보된 패턴을 Rule Book 승격 후보로 선별할 수 있음을 확인하였다

조합	건 수	액션 일치율	pseudo_code 일관성	대표 pseudo_code
EC2 + cost_inefficiency	3	100% (Resize)	67% (2/3 동일)	<pre> if cpu_utilization_mean < 5.0 and action_scores['Resize']['saving_rate'] > 0.5: return 'Resize'(target='one_tier_down')</pre>

			3/3)		
Lambda cost_spike	+	3	67% (Throttle 2/3)	33% (1/3 동 일)	if cost_mean > 1.0 and invocation_count_mean > 5000.0 and error_count_mean < 5.0: return 'Throttle'(ReservedConcurrentExecutions=10)
S3 risk_security	+	3	67% (Block 2/3)	33% (1/3 동 일)	(소수 케이스인 NoAction 예시)
RDS cost_inefficiency	+	3	100% (Resize 3/3)	33% (1/3 동 일)	if cpu_utilization_mean < 5.0 and action_scores['Resize']['saving_rate'] > 0.4: return 'Resize'(target=one_tier_down)

<표 4. 리소스와 이상 유형에 따른 pseudo code>

4.2.4. Action Agent

Decision Agent가 선택한 조치를 실제 AWS 리소스에 적용한다. 현재 EC2의 Stop-Resize, Lambda의 Throttle, Auto Scaling의 Scale In, S3의 Block 조치를 구현하였다. 실제 조치 수행 전에 대상 리소스의 상태를 스냅샷으로 저장하여 QA 실패 시 이전 상태로 복구할 수 있도록 구성하였다. AWS API 호출 과정에서 발생하는 오류는 예외 처리를 통해 실패 원인과 실행 결과를 기록하여 이후 QA 및 로깅 단계에서 확인할 수 있도록 하였다.

- **입력:** selected_action, resource_type, resource_id(EC2 면 InstanceId, Lambda 면 함수명, ASG 면 그룹명, S3 면 버킷명, 이것 그대로 boto3 파라미터에 씌), requires_approval, target_instance_type
- **처리:** requires_approval 이면 스킵, 아니면 take_snapshot(resource_type, resource_id)(boto3 describe_*로 조회) → execute_action(...)(resource_type+action 조합으로 분기해서 실제 boto3 호출)
- **출력:** pre_action_snapshot, action_executed, action_result

4.2.5. QA Agent

액션 실행 직후 조치 결과와 리소스 상태를 검증하여 자동 조치의 안전성을 확인한다. 액션 실행 후 일정 시간(300초) 대기하여 지표를 실제로 재조회한 뒤, 조치 결과와 리소스 상태를 검증하여 자동 조치의 안전성을 확인한다. 이상을 유발한 지표가 여전히 기준을 초과하지 않는지 확인하며, CPU와 같이 절대 임계값이 설정된 지표는 80%를 기준으로, 그 외 지표는 조치 전 기준선 대비 상대적 증가 여부를 기준으로 판단한다. 또한 조치 이후 비용이 일정 수준 이상 증가하지

않았는지와 선택한 액션이 정상적으로 수행되었는지를 함께 확인한다. 특히 선택

- **입력:** action_result, pre_action_snapshot, rollback_count, resource_id, resource_type, 관련 raw_metrics
- **처리:** 화이트리스트 체크 → 액션 후 300 초 대기 및 실측 지표 재조회 → 메트릭 변화 / 비용 변화 / 가용성 유지 3 개 SLA 체크 → 검증 실패 시 롤백 → Action 단계 재시도 → 롤백 횟수 2 회 초과 시 추가 실행 중단 및 Slack 알림 → 검증 통과 시 WAF Rate-based Rule 자동 해제
- **출력:** qa_passed, sla_check_result, rollback_count , whitelisted

된 액션이 해당 리소스 유형에 구현되어 있지 않아 실행 자체가 이루어지지 않은 경우에는 일반적인 실행 실패와 구분하여 검증 실패로 처리한다. 검증에 실패한 경우에는 사전에 저장한 조치 전 상태를 기준으로 롤백한 후 Action 단계를 재시도하며, 반복적인 실패가 발생하면 추가 실행을 중단하고 관리자에게 Slack 알림을 전달한다. 반대로 검증을 통과한 경우에는 AutoScaling EDoS 대응을 위해 적용된 WAF Rate-based Rule이 남아 있다면 자동으로 해제한다. 이를 통해 단순히 액션을 실행하는 데 그치지 않고 실행 결과와 리소스 상태를 실측 기반으로 검증하여 자동화 과정에서 발생할 수 있는 부작용을 최소화하도록 구성하였다.

4.2.6. Logging Agent

탐지부터 조치 및 검증까지의 전체 실행 과정을 PostgreSQL 기반 Audit Log에 기록한다. agent_runs에는 파이프라인 실행 단위의 정보를 저장하고, agent_steps에는 각 단계의 실행 결과를 기록하며, action_log에는 실제 액션이 수행된 경우 조치 전후의 리소스 상태와 성공 여부를 저장한다. 이를 통해 각 이상 상황에 대해 어떤 판단과 조치가 수행되었는지 추적할 수 있도록 구성하였다.

- **입력:** 파이프라인 전체 state
- **처리:** DB(agent_runs/agent_steps/action_log) 저장 + Rule Book 자동승격 트리거.
- **출력:** log_entries, DB 에 저장된 단계별 실행 이력

4.3. 이상 탐지 모델 설계 (Isolation Forest / Z-score / 고정임계값)

4.3.1. 데이터 준비

리소스 유형별(EC2/Lambda/S3/AutoScaling)로 CPU 사용률·네트워크 입출력·호출 횟수·에러 횟수·비용 등 지표를 30개 시점의 슬라이딩 윈도우로 구성하여 탐지 모델의 입력으로 사용한다. 초기 학습 데이터는 리소스 유형별 정상 시나리오를 반영하여 생성한 목업 데이터를 활용하였다. 이후 구축된 초기 모델의 탐지 성능은 시나리오별로 별도 생성한 라벨링된 목업 평가 데이터 (정상·이상·경계)을 이용하여 검증하였다. 구체적인 시딩 방식과 자기참조 학습 버퍼의 구성은 4.3.3절에서 설명한다.

초기 구현 과정에서는 사전 학습 데이터가 충분하지 않은 리소스 유형에서 콜드스타트로 인한 오탐 문제가 발생하였으며, 이에 대한 해결책으로 초기 시딩을 통해 구축한 모델을 고정하여 사용하고 있다. 현재는 운영 과정에서 새롭게 수집되는 실측 데이터를 자동으로 재학습에 반영하지 않는다.

EC2의 경우 인스턴스 나이(LaunchTime 이후 경과 시간)를 별도로 조회하여, 막기동된 신생 인스턴스가 초기 워밍업 구간의 불안정한 지표로 인해 오탐되지 않도록 나이 가드(age guard)를 적용하였다. 나이 조회에 실패하더라도 판단을 보류하지 않고 기존과 동일하게 평가하도록 하여, 부가 정보 조회 실패가 탐지 자체를 중단시키지 않도록 설계하였다. 이는 본 시스템 전반에 적용된 '부가 기능의 실패가 핵심 기능의 실행을 방해하지 않는다'는 방어적 설계 원칙의 한 사례이다. Slack 알림이 실패하더라도 파이프라인은 계속 진행되도록 설계한 것 역시 동일한 원칙에 해당한다(4.4.4절 참고).

4.3.2. 데이터 분석 및 하이퍼파라미터 값 선정

Isolation Forest는 contamination=0.1, 이상 판정 임계값 $\tau=0.5$ 를 최종적으로 적용하였으며, 이상 스코어가 해당 임계값을 초과하면 이상으로 판정한다. Z-score는 $k=2.75$ (약 99.4% 정상 범위를 벗어나는 값)을 최종 임계값으로 사용한다. contamination은 Isolation Forest 학습 과정에서 전체 데이터 중 이상치가 차지할 것으로 가정하는 비율을 의미하며, 값이 클수록 더 많은 관측치를 이상치로 간주하도록 모델의 판정 기준이 설정된다.

이러한 값들은 임의로 설정한 것이 아니라 4.3.4절에서 수행한 그리드 서치를 바탕으로 선정하였다. 초기 기본값인 $\tau=0.6$, $k=3.0$ 을 기준으로 여러 임계값 조합을 비교하였으며, 목표 정확도 80%를 기준으로 대비 재현율과 정밀도 간의 트레이드오프

를 함께 고려하여 최종값을 결정하였다.

한편 EC2 유휴 상태는 CPU 사용률과 네트워크 사용량이 지속적으로 낮은 상태에서 인스턴스 실행에 따른 비용이 계속 발생하는 경우를 의미한다. 이러한 상태는 급격한 지표 변화가 나타나지 않아 Z-score 기반 탐지에서 포착되지 않을 수 있으며, 여러 리소스 타입을 함께 학습하는 Isolation Forest에서도 다른 리소스의 낮은 지표값과 구분하기 어려울 수 있다.

실제로 학습된 Isolation Forest의 트리 100개를 분석한 결과, 상위 3단계 이내에서 리소스 타입 구분 컬럼을 기준으로 분할한 트리는 46개에 그쳤다. 즉, 나머지 트리는 리소스 타입을 먼저 구분하지 않은 상태에서 개별 지표값을 기준으로 데이터를 분할하였다. 이때 CPU 및 네트워크 지표가 존재하지 않는 다른 리소스 타입에서 해당 값을 0으로 채운 데이터가 전체 학습 데이터의 약 60%를 차지하기 때문에, 모델이 이러한 데이터와 실제 CPU·네트워크 사용량이 매우 낮은 EC2 유휴 상태를 유사한 패턴으로 인식할 가능성이 있다.

따라서 본 시스템에서는 Isolation Forest와 Z-score만으로 EC2 유휴 상태를 판정하지 않고, CPU 사용률과 네트워크 사용량에 대한 절대 임계값을 별도로 적용하는 보완 탐지 로직을 구성하였다.

4.3.3. 모델 학습용 데이터셋 구축 및 평가

Isolation Forest는 초기에는 리소스 유형별로 개별 모델을 학습하는 방식으로 구성하였으나, 이후 EC2, Lambda, Auto Scaling, S3의 4개 리소스 유형을 하나의 모델에서 학습하는 통합 모델 방식으로 전하였다. 또한 운영 과정에서 모델이 판정한 데이터를 다시 학습 버퍼에 반영하는 자기참조 학습 버퍼 구조를 도입하였다. 초기에는 정상 시나리오를 반영한 목업 데이터로 학습 버퍼를 시딩(seeding)하고, 이후 운영 데이터가 축적됨에 따라 자기참조 재학습 구조를 통해 기존 데이터를 점진적으로 대체하여 모델을 갱신할 수 있도록 설계하였다.

자기참조 재학습 구조는 모델이 정상으로 판단한 윈도우 중 정상성이 충분히 확보된 데이터만 학습 버퍼에 추가하고, 축적된 데이터를 이용하여 모델을 주기적으로 재학습하는 방식이다. 자기 참조 과정에서 잘못 판정된 데이터가 학습에 반영되는 것을 최소화하기 위해 이상 판정 기준보다 보수적인 채택 기준을 적용하였다. 이를 통해 정상과 이상의 경계에 위치한 데이터는 학습 대상에서 제외하고, 정상으로 판단할 신뢰도가 높은 데이터만 학습 버퍼에 반영하도록 하였다.

하나의 탐지 윈도우는 5분 간격으로 수집된 30개 시점의 지표로 구성되며,

Isolation Forest는 해당 윈도우 내 30개 시점의 이상 점수를 기반으로 이상 여부를 판단한다. 학습 버퍼는 각 리소스 유형 별로 최대 30개 윈도우만 유지하는 FIFO 방식으로 구성하였다. 새로운 정상 윈도우가 추가되면 가장 오래된 데이터를 제거함으로써 과거의 사용 패턴보다 최근 운영 환경의 특성을 지속적으로 반영하도록 하였다. 이 방식은 특정 시점에 잘못 판정된 데이터가 버퍼에 유입되더라도 그것이 무한하게 누적되어 모델에 영구적으로 남는 것은 방지하지만, 최근 연속으로 오판이 발생하는 경우 버퍼 전체가 오염된 데이터로 채워질 위험까지는 막지 못한다는 한계도 가진다.

모델 재학습은 새로운 정상 윈도우가 5개 누적될 때마다 수행하도록 구성하였다. 매 수집 주기마다 모델을 갱신하는 대신 일정량의 데이터를 묶어 반영함으로써 재학습에 필요한 자원 소모를 줄이고, 하나의 윈도우가 모델에 과도한 영향을 미치는 것을 완화하였다.

다만 본 보고서에서는 동일한 조건에서 재현 가능한 성능 평가를 위해 목업 데이터로 구성된 초기 학습 버퍼를 동결하였다. 따라서 본 보고서의 평가 결과는 운영 실측 데이터를 이용한 자기참조 재학습이 진행된 모델이 아니라, 동일한 초기 시딩 데이터를 기준으로 고정된 모델의 성능을 나타낸다. 향후 동결 설정을 해제할 경우 운영 환경에서 축적되는 실측 데이터를 학습 버퍼에 반영할 수 있도록 구성하였다.

모델 평가는 리소스 유형별로 구성된 정상(normal), 이상(anomaly), 경계(edge) 윈도우 데이터셋을 대상으로 탐지 성능을 측정하는 방식으로 수행하였다. 목업 평가 데이터셋을 이용한 결과는 다음과 같다.

리소스	정상 오탐률	이상 탐지율	경계(edge) 정상 오탐률
EC2 (좀비)	10.0% (2/20)	100% (10/10)	20.0% (1/5)
Lambda (스스로 재시도폭증)	25.0% (5/20)	100% (10/10)	40.0% (2/5)
AutoScaling (EDoS)	0.0% (0/20)	100% (10/10)	0.0% (0/5)
S3 (대량다운로드)	0.0% (0/20)	100% (10/10)	0.0% (0/5)

<표 5. 리소스별 목업 데이터 평가 결과>

아울러 SHAP 기반 해석가능성 검증에서는 리소스 유형별로 SHAP 기여도가 가장 높은 지표와 실제 이상을 주입한 지표가 일치하는 비율을 측정하였다. 그 결과

Auto Scaling 100%, Lambda 96.7%, EC2 93.3%, S3 71.1%의 일치율을 보였다[4]. 특히 S3의 일치율이 상대적으로 낮게 나타난 것은 대량 다운로드 시나리오에서 number_of_requests와 bytes_downloaded가 동시에 변화함에 따라 SHAP 기여도가 두 지표에 분산된 영향으로 추정된다. 다만 이는 결과에 대한 정성적 해석이며, 해당 원인에 대한 별도의 통계적 검증은 수행하지 않았다.

4.3.4. 최적화 (성능 최적화, 정확도 최적화 등 디버깅 과정)

파라미터 튜닝은 generate_eval_dataset.py로 생성한 합성 평가 데이터셋 435개를 기반으로 그리드 서치를 통해 수행하였다. 평가 데이터셋은 5개 리소스 유형별 정상·이상 시나리오로 구성하였으며, contamination=0.1을 고정한 상태에서 이상 판정 임계값 τ 19개(0.05~0.95)와 Z-score 임계값 k 13개(1.0~4.0)를 조합하여 총 247개 조합을 탐색하였다. contamination을 고정한 것은 윈도우 내부에서 이상 점수를 min-max 방식으로 정규화하는 현재 구조에서는 해당 값의 변화가 최종 판정 결과에 미치는 영향이 제한적임을 사전 실험을 통해 확인하였기 때문이다. 튜닝 전/후 대표 값 비교는 다음과 같다.

구분	contamination	τ (IForest)	k (Z-score)	정확도	정밀도	재현율	F1	Joint catch rate
튜닝 전 (기본값)	0.1	0.6	3.00	79.77%	84.08%	85.26%	84.67%	96.07%
튜닝 후 (최적값)	0.1	0.5	2.75	80.46%	79.94%	93.68%	86.27%	98.93%

<표 6. 튜닝 전후 결과 지표 비교>

그리드 서치 결과, 목표 정확도 80%를 상회하는 조합은 총 31개였으며, 이 가운데 정밀도와 재현율의 균형 및 이상 상황에 대한 탐지 성능을 종합적으로 비교하여 최종적으로 $\tau=0.5$, $k=2.75$ 를 선정하였다. 튜닝 전후의 대표 성능을 비교한 결과, 정밀도는 84.08%에서 79.94%로 소폭 감소하였으나 재현율은 85.26%에서 93.68%, Joint catch rate는 96.07%에서 98.93%, F1-score는 84.67%에서 86.27%로 향상되었다. 본 연구에서는 클라우드 비용 이상을 탐지하지 못하는 미탐(FN)이 불필요한 알람이 발생하는 오탐(FP)보다 더 큰 비용 손실로 이어질 수 있다고 판단하여, 정밀도의 일부 감소를 허용하더라도 재현율을 우선하는 방향으로 최종 파라미터를 선정하였다.

이 외에도 IForest 정규화 버그, 신규 리소스 콜드스타트 오탐, S3 반복시행 판정 로직 불일치 등 디버깅 이력은 5.5절에 정리하였다.

4.4. 자동화 대응 및 운영 시스템

4.4.1. 탐지-대응 파이프라인 (inbound 규칙, rule-book 도입)

본 시스템은 탐지된 이상 상황에 대해 분류, 조치 결정, 실행 및 검증을 자동으로 수행할 수 있도록 Rule Book 과 리소스별 대응 함수를 연계하여 구성하였다.

Rule Book 은 Classification, Decision, QA 단계에서 활용되는 규칙을 관리하며, 리소스 유형, 트리거 지표 및 시간대 조건 등에 따라 적절한 규칙을 매칭한다. 특히 time_window 조건을 통해 특정 시간대나 요일에 따라 규칙의 적용 여부를 판단할 수 있도록 구성하였다. 예를 들어 야간 배치 작업 시간대에는 일시적인 비용 증가를 정상적인 패턴으로 판단하도록 시간 조건을 적용할 수 있다.

Rule Book 은 고정된 규칙만을 사용하는 방식에서 벗어나 반복적으로 동일한 LLM 판단이 발생하는 패턴을 규칙으로 관리할 수 있도록 구성하였다. 동일한 판단이 일정 횟수 이상 반복되고 QA 를 통과한 패턴은 규칙 승격 후보로 등록되며, 최종 승격 여부는 관리자 승인을 거쳐 결정된다. 초기에는 조건을 만족한 패턴을 자동으로 Rule Book 에 등록하는 방식으로 설계하였으나, 자동화된 규칙이 잘못 생성될 경우 이후 동일한 상황에서 잘못된 판단이 반복될 가능성을 고려하여 관리자 승인 방식으로 변경하였다. 해당 변경은 중간보고서 이후 반영된 주요 설계 변경 사항이다.

실행 단계에서는 리소스 유형과 이상 유형에 따라 허용된 조치를 매핑하여 실제 AWS 리소스에 적용한다. 특히 비용 이상뿐만 아니라 대량 요청과 같은 보안·트래픽 이상에 대응할 수 있도록 인바운드 트래픽 제어 기능을 별도 모듈로 구성하였다. AWS Security Group 은 허용(allow) 규칙을 기반으로 접근을 제어하며 직접적인 차단(deny) 규칙을 제공하지 않으므로, 특정 트래픽의 차단이 필요한 경우 WAF Rate-based Rule 또는 Network ACL(NACL)의 deny 규칙과 같은 별도의 제어 수단이 필요하다. 본 시스템에서는 WAF Rate-based Rule 과 IP 블랙리스트를 활용하여 특정 출처의 과도한 요청이나 비정상 트래픽을 차단할 수 있도록 구현하였다. 모든 인바운드 제어 함수는 dry_run 옵션을 지원해 실제 API 호출 없이 '실행되었다면 어떤 결과였을지'를 먼저 검증할 수 있다.

리소스	anomaly_type	허용 액션	실제 실행 메커니즘
EC2	cost_inefficiency	NoAction / Stop /	Boto3의 stop_instances,

	(좀비/오버프로비저닝)	Stop+Schedule / Resize	modify_instance_attribute 등을 이용하여 조치 하며, 실행 전 상태를 저장하고 QA 실패 시 롤백
Lambda	cost_spike (호출·재시도 폭증)	NoAction / Throttle / Block	throttle_lambda_concurrency()를 통해 Reserved Concurrency를 조정하여 실행량을 제한하고, QA 검증 실패 시 _rollback_lambda()를 통해 스냅샷에 저장된 기존 동시성 설정으로 복원
AutoScaling	risk_security (EDoS)	NoAction / Block / Scale In	scale_down_with_rate_limit()을 통해 Auto Scaling 규모를 조정하고, 연결된 ALB에 WAF Rate-based Rule을 적용
S3	risk_security (대량 다운로드)	NoAction / Block	S3 Block 조치 및 WAF IP 블랙리스트를 적용 하여 비정상 접근을 제한

<표 7. 리소스별 이상 타입 및 액션과 액션에 따른 실제 실행 매커니즘>

Decision Agent의 조치 선택 방식은 중간보고서 이후 변경되었다. 초기에는 saving rate, impact score 및 stability score를 결합한 고정 점수식을 이용하여 후보 액션을 선택하였으나, 현재는 LLM이 각 액션의 AWS API 사양, 비용 효과, 예상 부작용 및 복구 가능성을 함께 고려하여 최종 액션을 선택하도록 변경하였다. 이 과정에서 비용 절감 효과와 관련된 saving_rate 및 estimated_saving_usd는 LLM이 임의로 추정하지 않도록 원시 비용 지표를 기반으로 사전에 계산하여 프롬프트에 제공한다. 예를 들어 Stop은 해당 비용의 100% 절감, Stop+Schedule은 50%의 가동 시간 감소를 가정하며, Resize는 적용 가능한 인접 인스턴스 유형 간 비용 차이를 기반으로 절감액을 계산한다. 이를 통해 수치 계산은 결정론적으로 처리하고, LLM은 서비스 영향과 복구 가능성 등을 고려한 조치 선택에 집중하도록 구성하였다.

조치의 위험도는 anomaly_type 별 기본 위험도를 기준으로 결정하며, 선택된 액션에 따라 최소 위험도를 상향 조정한다. cost_inefficiency는 LOW, cost_spike는 MED, risk_security는 HIGH를 기본 위험도로 적용하고, Resize 및 Stop+Schedule은 최소 MED, Block은 최소 HIGH로 설정하였다. MED 이상의 조치는 관리자 승인이 필요한 대상으로 분류하여 승인 게이트를 거치도록 구성하였다.

Action 이후에는 QA 단계를 통해 자동화된 조치의 결과를 검증한다. QA에서는 CPU 사용률 80% 이하, 조치 전후 비용의 10% 이상 증가 여부 및 액션의 정상 완료 여부를 기준으로 조치 결과를 확인한다. 검증에 실패한 경우 rollback_count를 확인하여 최대 2회까지 후속 조치를 재시도하고, 반복적인

실패가 발생하면 관리자에게 상황을 통지한 후 실행 이력을 기록한다. 롤백이 필요한 경우에는 각 리소스 유형에 대해 조치 실행 전에 저장한 상태 정보를 기준으로 원상 복구를 수행하도록 구성하였다.

4.4.2. 가중치 조절 메커니즘 (관리자 설정 → LLM 프롬프트 동적 반영)

본 시스템은 관리자가 비용 절감과 서비스 가용성 사이의 우선순위를 직접 설정할 수 있도록 정책 설정 기능을 구현하였다. 관리자 웹 화면에서 '가용성 우선', '균형', '비용 절감 우선' 중 하나의 정책을 선택하면 해당 설정이 파이프라인의 Decision Agent 에 반영되어, 동일한 이상 상황에서도 관리자가 설정한 우선순위에 따라 조치 선택 기준이 달라지도록 구성하였다.

관리자 웹 프론트엔드의 Settings 화면에서는 세 가지 정책을 버튼 형태로 제공한다. 사용자가 정책을 선택하면 각 정책에 대응하는 `priority_weight` 값이 FastAPI 의 `/settings` API 를 통해 전달되며, 변경된 값은 `config/decision_policy.json` 에 저장된다. 이를 통해 관리자가 설정한 정책을 관리자 화면에만 표시하는 것이 아니라 파이프라인에서 실제 의사결정에 활용할 수 있도록 구성하였다.

파이프라인 워커와 관리자 API 는 서로 독립된 프로세스로 실행되므로 메모리 변수만으로 설정값을 공유할 수 없다. 이에 따라 두 프로세스가 공통으로 접근할 수 있는 JSON 파일을 설정 저장소로 활용하였다. Decision Agent 는 조치를 결정하는 시점에 저장된 `priority_weight` 값을 읽고, 해당 값에 따라 가용성 우선, 균형, 비용 절감 우선에 해당하는 지침을 생성하여 LLM 프롬프트에 반영한다.

'가용성 우선' 정책에서는 서비스에 미치는 영향이 큰 조치를 가급적 피하고 상대적으로 영향이 적은 조치를 우선하도록 하며, '균형' 정책에서는 비용 절감 효과와 서비스 영향 사이의 균형을 고려하도록 한다. '비용 절감 우선' 정책에서는 서비스 가용성에 치명적인 영향이 없는 경우 비용 절감 효과가 높은 조치를 우선하도록 구성하였다.

이와 같이 관리자가 선택한 정책은 단순히 화면에 표시되는 설정값에 그치지 않고 실제 Decision Agent 의 프롬프트 구성과 조치 선택에 반영된다. 따라서 관리자는 동일한 이상 상황에 대해서도 비용 절감과 서비스 가용성 중 어느 가치를 우선할지 세 가지 정책 중 선택하여 시스템의 대응 방향을 조정할 수 있다.

4.4.3. 승인 대기 메커니즘 (LangGraph interrupt + PostgreSQL 체크포인트)

중간보고서 시점에는 위험도가 높은 조치에 대해 pending_approval 상태만 기록하고 파이프라인 실행을 종료하는 방식으로 구현되어 있었다. 이 방식은 관리자의 승인 여부를 기록할 수 있지만, 승인 이후 중단된 실행을 원래 위치에서 재개할 수 없다는 한계가 있었다. 이를 개선하기 위해 현재 시스템에서는 Decision 과 Action 사이에 승인 게이트를 추가하고, LangGraph 의 실행 중단 및 상태 저장 기능을 활용하여 관리자 승인 이후 중단된 파이프라인을 이어서 실행할 수 있도록 구현하였다.

MED 이상의 위험도를 가진 조치가 승인 대상으로 판단되면 approval_gate 에서 파이프라인 실행을 일시 중단하고, Checkpointer 를 통해 중단 시점의 State 를 PostgreSQL 에 저장한다. 관리자가 승인 또는 거부를 수행하면 저장된 실행 상태를 기반으로 중단된 지점부터 파이프라인을 재개한다. 이를 통해 관리자의 승인에 시간이 소요되더라도 기존 실행 상태와 판단 결과를 유지할 수 있도록 구성하였다.

관리자가 조치를 승인한 경우 Action 단계로 진행하여 실제 AWS 리소스에 대한 조치를 수행한다. 반대로 조치를 거부한 경우에는 선택된 액션을 NoAction 으로 변경하여 실제 조치를 수행하지 않고 이후 로깅 단계로 진행한다. 따라서 승인 여부가 단순히 기록되는 것이 아니라 실제 파이프라인의 실행 여부를 결정하도록 구성하였다.

관리자 웹에서는 승인 대기 중인 실행을 조회하고 승인 또는 거부할 수 있는 '승인 대기 큐'를 제공한다. 승인 요청 처리 시 해당 실행이 실제 승인 대기 상태인지 확인하여 이미 처리되었거나 유효하지 않은 요청은 실행되지 않도록 방어 로직을 적용하였다.

이와 같은 승인 대기 메커니즘을 통해 본 시스템은 위험도가 낮은 조치는 자동으로 수행하는 동시에, 서비스에 미칠 영향이 큰 조치는 관리자의 판단을 거치도록 하였다. 또한 승인 이후에도 기존 실행 상태에서 파이프라인을 재개할 수 있도록 구성하여, 자동화의 효율성과 관리자의 통제권을 함께 확보하였다.

4.4.4. 관제 및 알림 시스템

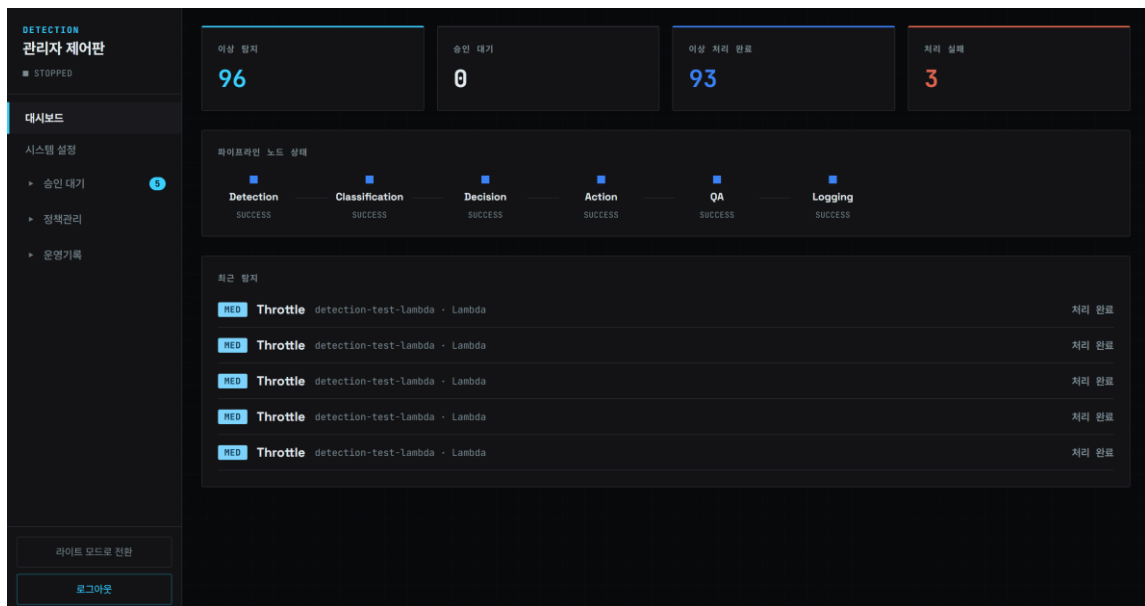
본 시스템은 자동화된 파이프라인의 동작상태를 관리자가 실시간으로 확인하고 필요에 따라 개입할 수 있도록 웹 제어판(설정용), Grafana 대시보드(확인용), Slack 알림

이라는 3 개의 관제 기능을 구성하였다.

4.4.4.1. 관리자 웹 제어판

관리자 웹 제어판은 관리자가 파이프라인 상태를 확인하고 시스템 설정값들을 변경하며 직접 개입할 수 있는 관리 창구이다. FastAPI 백엔드(api/)와 React 프론트엔드(frontend/)로 구성되며, 화면은 대시보드, 시스템 설정, 승인 대기, 정책 관리, 운영 기록의 5개 탭으로 구성된다. 모든 탭의 UI는 실제 백엔드 데이터와 연동되어 있다. 대시보드의 통계와 파이프라인 노드 상태는 실행 중인 파이프라인의 실제 state를, 설정 화면의 값들은 실제로 탐지 파이프라인에 적용되는 설정값을, 로그·이력 화면은 PostgreSQL과 JSONL 로그 파일에 실제로 기록된 데이터를 그대로 보여준다. 전체 시스템(PostgreSQL, FastAPI 백엔드, React 프론트엔드)은 `start.ps1` 쉘 스크립트 하나로 일괄 실행할 수 있도록 구성하였다.

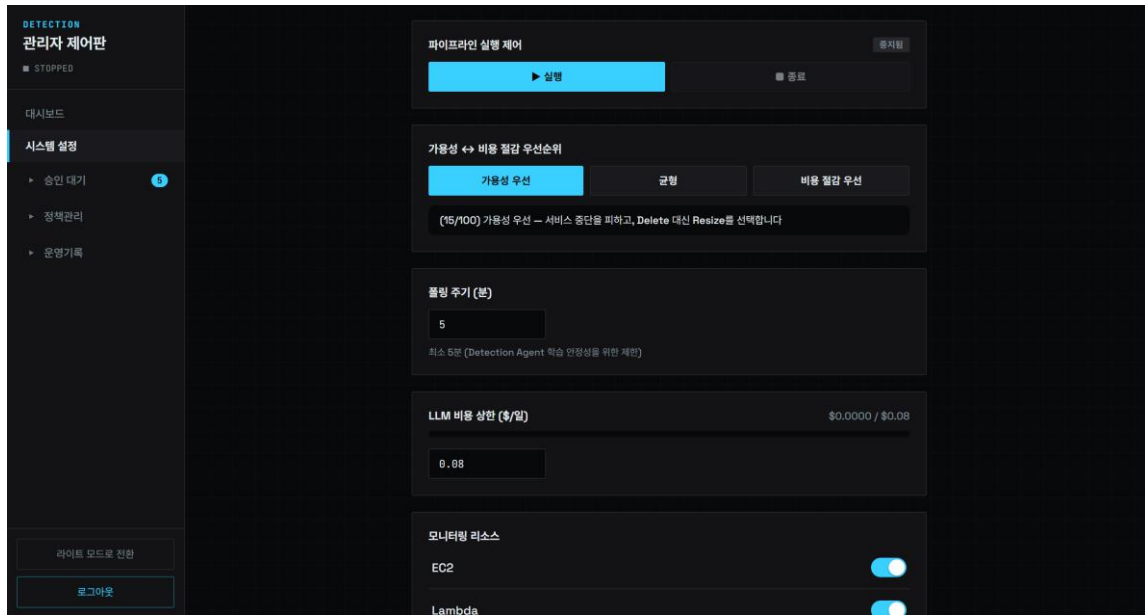
1. 관리자 제어판 - 홈 대시보드



<그림 2. 관리자 제어판 - 홈 화면(대시보드)>

화면 대시보드 탭에서는 이상 탐지, 승인 대기, 이상 처리 완료, 조치 실패 건수를 카드 형태로 표시한다. 또한 파이프라인 6단계의 현재 상태를 노드 단위로 시각화하며, 최근 탐지된 이상 목록을 함께 제공한다.

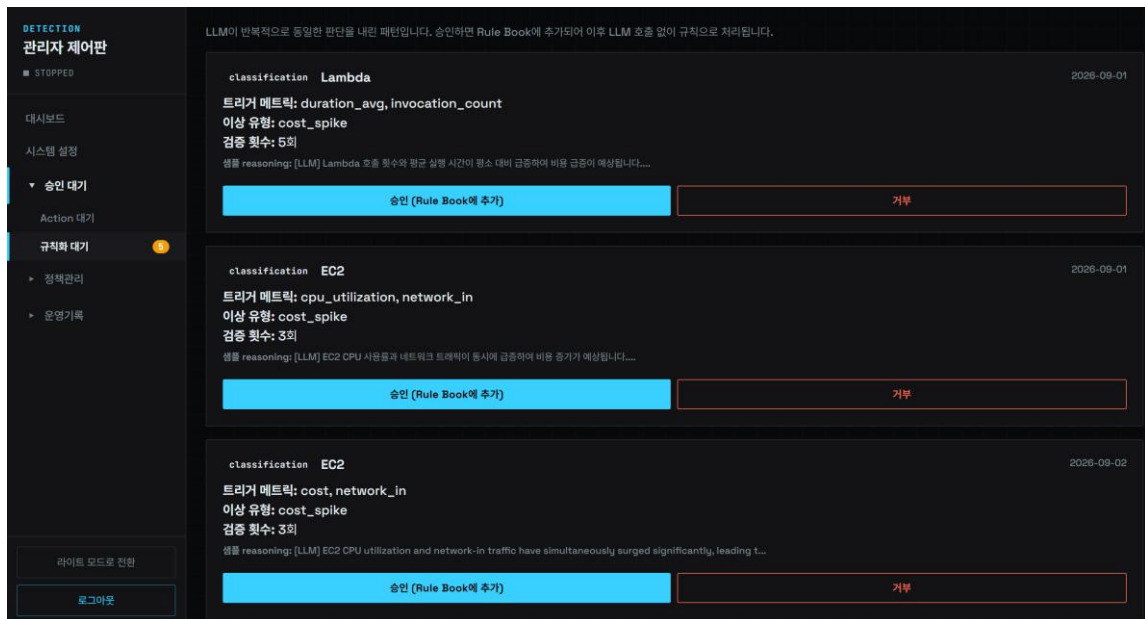
2. 관리자 제어판 - 시스템 설정



<그림 3. 관리자 제어판 - 시스템 설정 화면>

시스템 설정 탭에서는 폴링 주기와 리소스 유형별 탐지 활성화 여부를 설정할 수 있으며, 저장된 값은 실제 탐지 파이프라인에 반영된다. 폴링 주기는 담당자 요청에 따라 5분 미만으로는 설정할 수 없도록 서버 측에서 제한하였다. 이는 탐지 모델이 슬라이딩 윈도우와 베이스라인을 계산하기 위해 일정 수준 이상의 관측 간격을 필요로 하기 때문이다. 또한 파이프라인 프로세스를 관리자가 웹 제어판에서 직접 시작하거나 종료할 수 있도록 시스템 entrypoint를 CLI가 아닌 GUI 환경에서 호출되는 구조로 구성하였다. 이를 통해 별도의 터미널 조작 없이 웹 제어판에서 파이프라인의 실행 상태를 관리할 수 있도록 하였다. 파이프라인 실행 중에는 설정 변경을 서버 측에서 차단하여 운영 중 설정 변경으로 인한 예기치 않은 동작을 방지하며, 현재 설정값을 YAML 파일로 내보내 백업하거나 공유할 수 있도록 구성하였다.

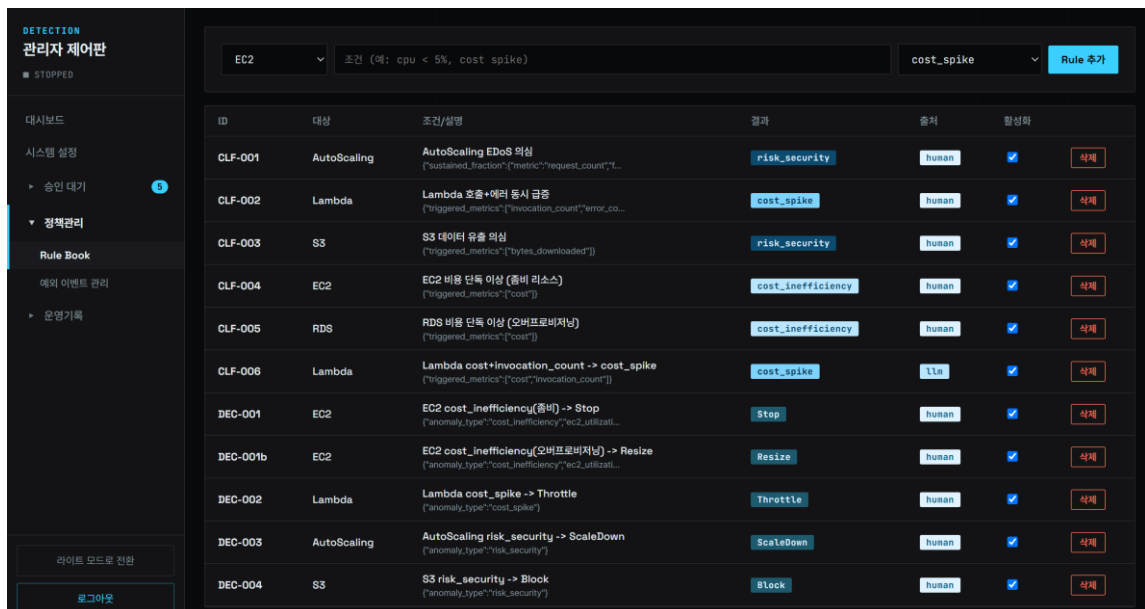
3. 관리자 제어판 – 규칙화 대기



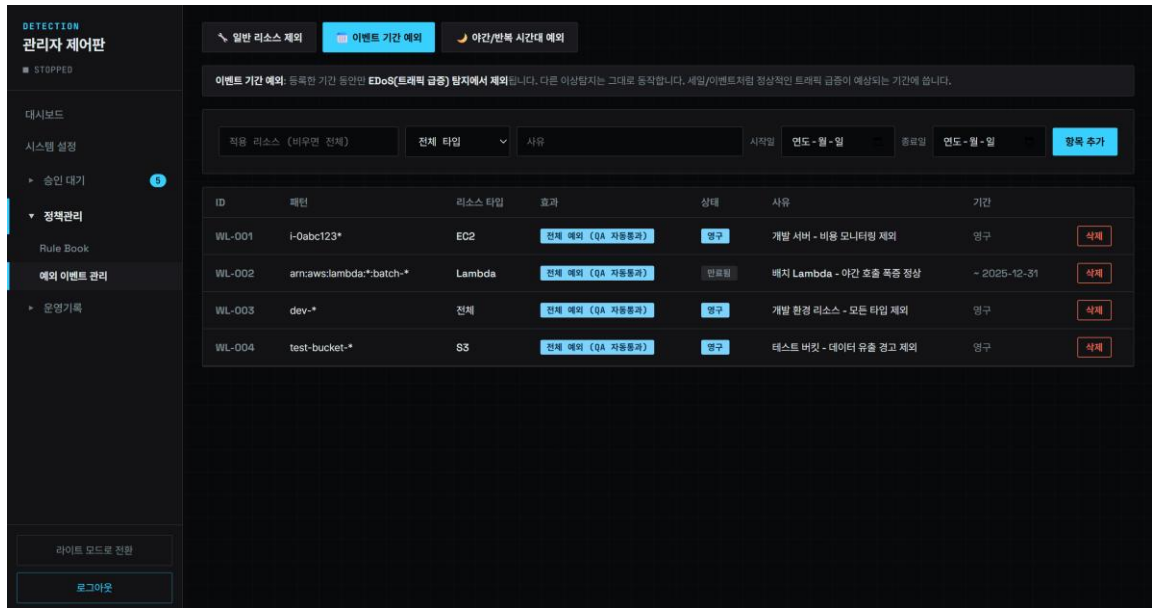
<그림 4. 관리자 제어판 - 규칙화 대기 화면>

승인 대기 탭에서는 위험도가 높아 사람의 승인이 필요한 조치 목록과, 반복적으로 발생한 LLM 판단을 Rule Book의 규칙으로 승격할지 검토하는 목록을 각각 제공한다.

4. 관리자 제어판 – 정책 관리



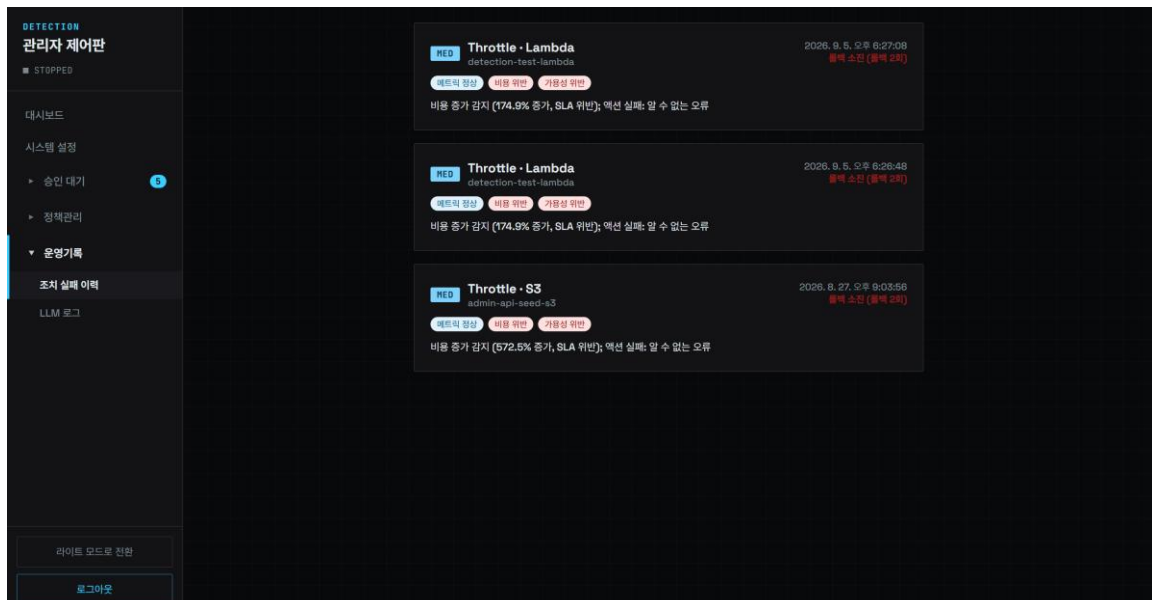
<그림 5. 관리자 제어판 - 정책 관리 중 Rule Book 화면>



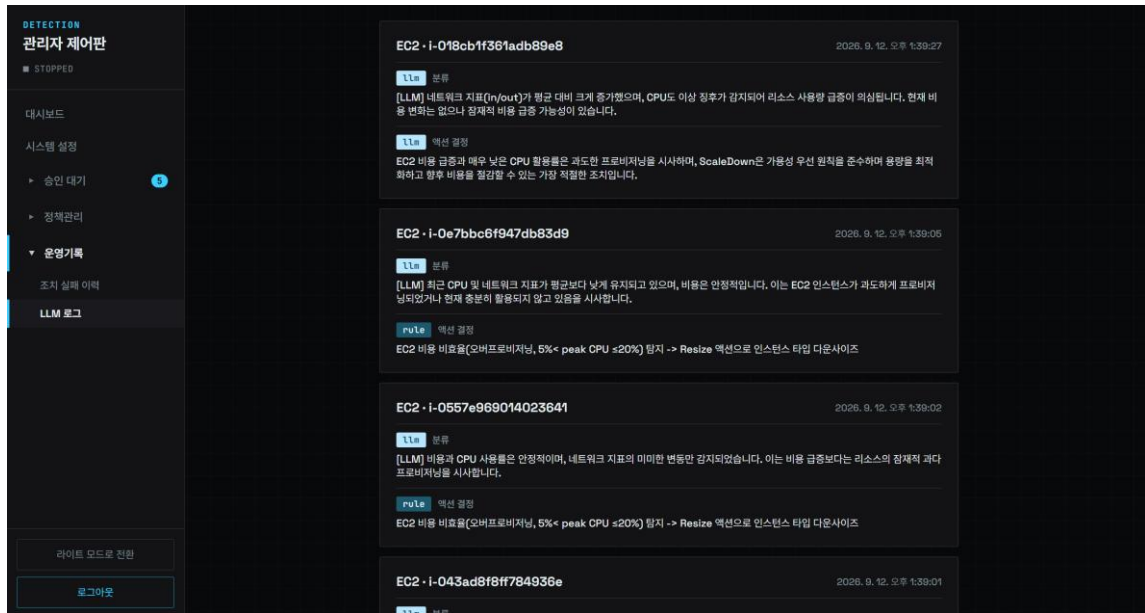
<그림 6. 관리자 제어판 - 정책 관리 중 예외 이벤트 관리 화면>

정책관리 탭에서는 Rule Book에 등록된 분류·판단 규칙과 예외 이벤트를 관리할 수 있다. 예외 이벤트는 일반 리소스 제외, 이벤트 기간 예외, 야간 및 반복 시간대 예외의 세 가지 모드로 구분하며, 관리자가 각 항목을 직접 조회·추가·삭제할 수 있도록 구성하였다.

5. 관리자 제어판 – 운영 기록



<그림 7. 관리자 제어판 - 운영 기록 중 조치 실패 이력 화면>



<그림 8. 관리자 제어판 - 운영 기록 중 LLM 로그 화면>

운영기록 탭에서는 조치 실패 이력과 LLM 판단 로그를 구분하여 제공한다. 여기서 "실패"로 기록되는 경우는 크게 세 가지이다. 첫째, 액션 실행 자체가 실패한 경우이다. 둘째, 액션은 정상적으로 실행되었으나 조치 후 재조회한 실측 지표가 메트릭·비용·가용성 SLA 중 하나라도 충족하지 못하여 문제가 실제로는 해소되지 않은 경우이다. 셋째, Rule Book의 QA 규칙에 의해 해당 조치가 명시적으로 실패로 판정된 경우이다. 이를 통해 자동화가 실패한 지점과 LLM이 실제로 수행한 판단을 사후에 추적할 수 있도록 하였다.

보안 관련해서는 다음과 같은 기능을 구현하였다.

보안 항목	구현 내용	파일
비밀번호 저장	bcrypt로 해시하여 저장	api/admin_users.py
인증 토큰 관리	- 로그인 성공 시 secrets.token_urlsafe로 추측 불가능한 무작위 세션 토큰 발급 - 서버 메모리에서 만료 시간과 함께 관리 - 이후 모든 요청은 X-Admin-Key 헤더로 검증	api/auth.py
SQL 인젝션 방지	- 데이터베이스 접근 전부 파라미터 바인딩(%s 플레이스홀더) 사용 - 문자열 조합 쿼리 없음 확인	pipeline/logging_agent.py 등
로그인 시도 제한	계정 단위로 5회 실패 시 5분 잠금	api/auth.py

입력 검증	모든 API 요청 본문을 Pydantic 모델로 스키마 검증	api/schemas.py
CORS 설정	프론트엔드가 실행되는 origin(localhost:3000)만 허용	api/main.py
환경 변수 관리	AWS 자격 증명 등을 .env 파일로 분리하고 .gitignore에서 제외	.env, .gitignore
로그 내 민감정보	로그에 비밀번호, 토큰 값 자체가 출력되지 않음을 확인	

<표 8. 관리자 제어판에 구현된 보안 관련 기능>

4.4.4.2. Grafana 대시보드

Grafana 대시보드는 시스템의 정량적 성과를 시각화하며, PostgreSQL Audit Log를 데이터소스로 활용한다. 비용 관점에서는 시스템 미적용 시 예상 비용과 시스템 적용 시 실측 비용을 30일 기준으로 정규화하고, 두 값의 차이로 산출한 비용 절감액을 함께 제시하여 시스템 적용에 따른 비용 절감 효과를 직관적으로 확인할 수 있도록 하였다. 이외에도 전체 시스템 정확도, 이상 유형별 분포, 자동 조치 이력, 파이프라인 성공률 및 실행 시간 등의 운영 지표를 패널로 구성하였다.



<그림 9. 그래파나 대시보드 예시 화면>

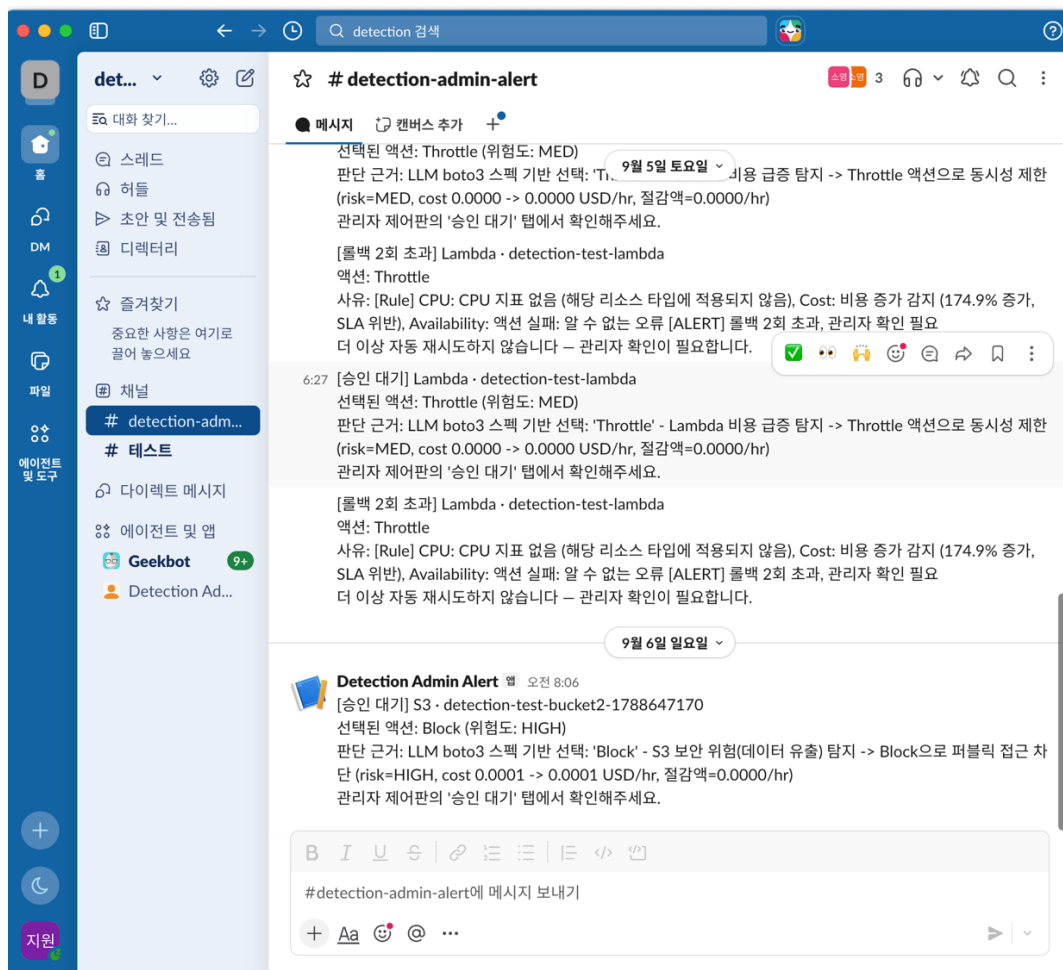
4.4.4.3. Slack 알림

Slack 알림은 사람의 판단이나 확인이 필요한 두 시점에 트리거되도록 설계하였다. utils/slack_notifier.py의 send_slack_alert()가 담당하며, (1) requires_approval=True로 승인 대기 상태가 발생한 경우와 (2) QA 과정에서 롤백이 2회를 초과하여 더 이상 자동 재시도를 수행하지 않는 경우에 전송된다.

알림은 decision(액션 결정) 직후, approval_gate(실제 중단) 직전에 위치한 notify_gate 노드에서 전송하도록 구성하였다. 이는 LangGraph interrupt()가 호출된 지점에서 그래프가 중단된 후 재개될 경우, 해당 노드의 함수가 다시 실행될 수 있다는 특성을 고려한 것이다. 만약 알림 전송 코드를 approval_gate_node 내부의

interrupt() 호출 이전에 배치할 경우, 최초 승인 대기 시 한 차례 알림이 전송된 후 관리자의 승인으로 파이프라인이 재개될 때 동일한 알림이 다시 전송될 수 있다. 이를 방지하기 위해 알림 전송과 interrupt() 호출을 서로 다른 노드로 분리하였다. 이에 따라 파이프라인 재개 시 이미 실행을 완료한 notify_gate는 다시 실행되지 않고, 승인 게이트부터 파이프라인이 재개되도록 구성하였다.

또한 Slack 웹훅 전송이 실패하더라도 웹훅 만료나 네트워크 오류 등의 예외가 파이프라인 실행에 영향을 주지 않도록 처리하였다. 웹훅 전송 과정에서 발생한 예외는 파이프라인 외부로 전파하지 않고 콘솔에 오류 원인만 기록하도록 하였다. 이때 requests 예외의 문자열 표현에 웹훅 URL이 포함될 수 있다는 점을 고려하여, 오류를 로그에 기록하기 전에 해당 URL 문자열을 치환·마스킹하도록 구현하였다. 이를 통해 알림 기능의 장애가 핵심 파이프라인의 실행을 중단시키지 않으면서도, 오류 원인을 확인할 수 있도록 하였다.



<그림 10. Slack 알림 예시 화면>

4.5. 전체 파이프라인 End-to-End 검증

승인 대기와 롤백 등 상대적으로 발생 빈도가 낮은 조건부 분기가 설계된 흐름에 따라 동작하는지를 별도로 검증하였다. 실제 AWS 호출이 필요하지 않은 조건부 분기는 해당 노드 함수를 연결한 테스트 그래프로 검증하였으며, 외부 시스템에 영향을 미치는 Slack 알림 등의 부작용은 대체(mock)하여 검증하였다.

검증 결과, 승인 및 롤백을 포함한 파이프라인의 주요 제어 흐름이 설계된 조건에 따라 동작함을 확인하였다.

조건분기 / 액션	검증 방식	결과
승인 → 액션 실행	통합테스트(checkpointer+interrupt 그래프)	통과
승인 거부 → NoAction	통합테스트	통과
QA 통과 → 로깅	실측(5 / 5)	통과
QA 실패 → 롤백 → 재시도	통합테스트 + 실측(AutoScaling WAF 8/8)	통과
롤백 2회 초과 → 포기 + Slack 알림	통합테스트	통과
EC2 Resize	실측(5건)	통과
Lambda Throttle	실측(8건)	통과
Lambda Block	실측	통과
AutoScaling Scale In	실측(8건)	통과
S3 Block	실측(3건)	통과
EC2 Stop	실측(5건, v3 버그수정판)	통과
AutoScaling/EC2 Block(WAF)	실측(8건)	통과

5. 연구 결과 분석 및 평가

5.1. 시나리오 설계 개요

평가는 리소스 유형과 이상 유형을 조합한 5개 시나리오로 설계하였다: (1) EC2 좀비 리소스(cost_inefficiency), (2) EC2 오버프로비저닝(cost inefficiency) (3) Lambda 스로틀 재시도 폭증 (cost_spike), (4) AutoScaling EDoS 의심 트래픽(risk_security), (5) S3 대량 다운로드(risk_security). 각 시나리오는 탐지→분류→결정→실행→QA→로깅 전 구간을 end-to-end로 실행하며, 실제 AWS 리소스(EC2 13대, Lambda 13개 함수)를 대상으로 반복 시행(각 정상 8회, 이상 5~15회 내외)하여 측정하였다.

5.2. 시나리오별 실행 결과

본 절에서는 앞서 설계한 5개 시나리오를 대상으로 실제 AWS 환경에서 실제 값과 트래픽을 활용하여 파이프라인을 실행하고, 탐지부터 조치, QA, 로깅까지의 전체 실행 결과를 확인하였다.

시나리오	리소스	데이터 성격	선택 액션	QA 통과율
① EC2 좀비 리소스	EC2	실 AWS 13대 (anomaly 5 + normal 8)	Stop	100% (5/5)
② EC2 오버프로비저닝	EC2	실 AWS 13대 (anomaly 5 + normal 8)	Resize	100%(5/5)
③ Lambda 스로틀 재 시도 폭증	Lambda	실 AWS 13개 함수	Throttle	100%(8/8)
④ S3 대량 다운로드	S3	실 AWS 버킷 대상 실제 다운로드 트래픽 발생	Block	100%(3/3)
⑤ AutoScaling EDoS	AutoScaling (ALB+ASG)	실 AWS 트래픽 발생 (anomaly 15 + normal 8, n=23)	Scale In (+WAF)	100%(8/8)

<표 9. 시나리오별 AWS 실연동 실행 결과>

5.3. 시스템 성능

본절의 정량적 성능 지표는 산학협력 멘토 자문에서 지적된 정성적 검증에 그친다는 의견을 반영하여, 라벨링된 실측 데이터 기반의 정량 지표로 보강하였다.

5.3.1. 파이프라인 전체 성능

본 시스템의 파이프라인은 탐지(Detection) → 분류(Classification) → 결정(Decision) → 승인 대기(Approval Gate) → 조치(Action) → 검증(QA)의 순차적인 단계로 구성되며, 각 단계는 이전 단계를 통과한 케이스만 다음 단계로 전달되는 게이트(gate) 구조를 갖는다. 따라서 하나의 이상 케이스가 탐지부터 QA까지 전 과정을 정확하게 처리할 확률을 전체 정확도로 정의할 수 있으며, 각 단계가 독립적으로 실패할 수 있는 관문이라는 전제하에 단계별 정확도 또는 성공률의 곱으로 산출한다.

전체 정확도

= 탐지 정확도 × 분류 정확도 × 결정 정확도 × 실행 성공률 × QA 통과율

5.3.1.1. Detection Agent (탐지) 성능

탐지 성능은 confusion matrix(TP/FN/FP/TN)를 기반으로 산출한다. TP(True Positive)는 실제 이상을 이상으로 정확히 판정한 경우, FN(False Negative)은 실제 이상을 정상으로 판정하여 이상을 놓친 경우, FP(False Positive)는 정상 데이터를 이상으로 판정한 경우, TN(True Negative)은 정상 데이터를 정상으로 정확히 판정한 경우를 의미한다. 이 네 가지 값은 이후 제시하는 Accuracy, Recall, Precision, FPR, F1-score 등의 성능지표를 산출하는 기초값이며, 이항검정 및 McNemar 검정 역시 이를 기반으로 수행한다.

일반적으로 TP와 TN은 클수록, FN과 FP는 작을수록 바람직하다. 특히 본 연구에서는 비용 이상을 놓치는 FN의 영향을 중요하게 고려하므로, FN을 낮게 유지하는 것을 주요 평가 기준으로 설정하였다. 이는 FN과 FP가 이후 파이프라인에서 처리될 수 있는 기회에 차이가 있기 때문이다. FP(정상 데이터를 이상으로 오탐한 경우)는 이상으로 판정된 이후 분류·결정·실행 단계를 거치게 되며, 조치가 수행된 경우에도 QA 단계에서 실측 결과를 재검증하여 SLA 위반이 확인되면 자동 롤백을 수행할 수 있다. 따라서 FP는 이후 단계에서 오류를 감지하고 복구할 수 있는 기회가 존재한다.

반면 FN(이상을 정상으로 판정하여 놓친 경우)은 탐지 단계에서 이상으로 판정되지 않기 때문에 이후의 분류·결정·실행·QA 단계로 진행되지 않는다. 즉, 탐지 이후 단계에서 이를 추가로 검출하거나 조치할 기회가 없으므로 비용 이상이 그대로 방치될 수 있다. 따라서 본 연구에서는 FP보다 FN을 더 중요하게 고려하며, 탐지 단계에서 FN을 낮게 유지하는 것을 중점적인 평가 기준으로 설정하였다.

Accuracy·Recall·Precision·F1-score는 0~1(또는 0~100%) 범위에서 높을수록 바람직하며, FPR(False Positive Rate)은 실제 정상 데이터 중 이상으로 잘못 판정한 비율을 의미하므로 낮을수록 바람직하다.

$$\text{Accuracy(탐지 정확도)} = (TP + TN) / (TP + FN + FP + TN)$$

$$\text{Recall(재현율)} = TP / (TP + FN)$$

$$\text{Precision(정밀도)} = TP / (TP + FP)$$

$$\text{FPR(False Positive Rate)} = FP / (FP + TN)$$

Precision과 Recall은 일반적으로 트레이드오프 관계에 있어 재현율을 높이는 과정에서 정밀도가 낮아질 수 있으므로, 둘 중 하나의 지표만으로 탐지 방식의 성능을 판단하는 데에는 한계가 있다. 이를 보완하기 위해 두 지표의 조화평균인 F1-score를 함께 제시한다.

$$F1 - score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

전체 5개 시나리오에서 Recall은 66.7%~100% 범위로 나타났다. EC2 좀비와 EC2 오버프로비저닝은 Recall 100%로 가장 높게 나타났는데, 두 시나리오가 각각 CPU≤5%, CPU≤20%의 고정 임계값 기반 탐지를 사용하기 때문이다. 이는 통계·ML 기반 판정과 달리 측정값이 설정된 임계값을 만족하는지를 직접 확인하는 결정론적 방식이므로, 테스트 케이스가 임계값에서 충분히 떨어져 있을 경우 높은 Recall이 나타날 수 있다. 따라서 두 시나리오의 Recall 100%를 일반적인 탐지 성능의 우수성으로 과대 해석하기보다는, 임계값 경계에 가까운 사례가 충분히 포함되지 않았다는 점을 함께 고려할 필요가 있다. 반면 S3·Lambda·AutoScaling EDoS는 Z-score와 Isolation Forest 기반으로 이상 여부를 판정하므로 통계·ML 기반 탐지 성능을 평가하는 사례로 볼 수 있다. 이 중 표본 수가 가장 큰 AutoScaling EDoS(n=23)의 Recall 66.7%가 통계 기반 탐지 결과를 상대적으로 잘 보여주는 수치로 판단된다.

시나리오	TP	FN	FP	TN	Accuracy	Recall	F1-score	n(정상/이상)
EC2 좀비	5	0	0	8	100%	100%	1.000	8 / 5

EC2 오버프로비저닝	5	0	0	8	100%	100%	1.000	8 / 5
Lambda 스로틀 재시도폭증	5	0	3	5	76.9%	100%	0.769	8 / 5
S3 대량다운로드	5	0	1	7	92.3%	100%	0.909	8 / 5
AutoScaling EDoS	10	5	1	7	73.9%	66.7%	0.769	8 / 15

<표 10. 시나리오별 AWS 실연동 실행 결과 분석 지표>

5.3.1.2. Classification Agent (분류) 성능

분류 성능은 탐지된 이상 케이스에 대해 시스템이 부여한 anomaly_type이 실험 설계상의 정답 유형과 일치하는지를 기준으로 평가한다. 분류는 Rule Book 규칙을 먼저 적용하고, 규칙만으로 판단하기 어려운 경우에만 LLM에 위임되는 배타적 경로 구조이므로, 각 시나리오가 실제로 거친 경로(Rule Book 또는 LLM)를 함께 표기한다.

$$\text{분류 정확도} = \text{정답 anomaly_type과 일치한 분류 건수} / \text{전체 분류 시도 건수}$$

5.3.1.3. Decision Agent(결정) 성능.

결정 성능은 분류 결과를 바탕으로 Decision Agent가 해당 리소스 유형에 대해 실제로 구현되어 있어 실행 가능한 액션을 올바르게 선택했는지를 기준으로 평가한다. 이는 분류 단계의 정확도와 독립적으로 평가되는 지표이다. 즉, 분류 결과가 올바르더라도 Decision Agent가 리소스 유형을 적절히 고려하지 못해 해당 리소스에 구현되지 않은 액션을 선택한 경우에는 결정 단계의 오류로 평가한다.

$$\text{결정 정확도} = \text{리소스 유형별 구현된 유효 액션 집합에 포함되는 액션을 선택한 건수} / \text{전체 결정(액션 선택) 시도 건수.}$$

5.3.1.4. Action Agent (액션) 성능

조치 성능은 결정(Decision) 단계에서 유효한 액션으로 선택된 조치가 실제 AWS 리소스에 대해 성공적으로 실행되었는지를 기준으로 평가한다. 결정 단계에

서 해당 리소스에 구현되지 않은 액션으로 판정된 경우에는 이미 결정 단계의 오류로 분류되므로, 실행 성공률의 분모에서 제외한다. 이는 동일한 오류를 결정 단계와 실행 단계에서 중복하여 반영하지 않기 위함이다. 또한 승인이 필요한 액션이 Approval Gate에서 승인 대기 상태로 남아 측정 기간 내에 승인 또는 거절되지 않은 경우에는 실제 조치가 실행되지 않았으므로 실행 성공률의 분모에서 제외한다.

실행 성공률

= 실제 실행이 시도된 유효 액션 중 success 건수
/ 유효하게 결정되어 실제 실행이 시도된 건수

QA Agent (검증) 성능

QA 통과율 = qa_passed = True 인 건수 / QA 검증이 수행된 전체 건수

QA 성능은 조치 실행 이후 시스템이 SLA(가용성·메트릭 복구 등)를 실측하여 재검증하고 정상 판정(qa_passed=True)했는지를 기준으로 평가한다. QA가 조치 실패나 SLA 위반을 감지하면 자동 롤백이 트리거되며, 이 경우 qa_passed=False로 기록된다.

QA 통과율은 일반적으로 높을수록 좋지만, 통과율이 낮다고 해서 항상 시스템의 성능이 낮다는 의미는 아니다. 조치 자체의 실패를 QA라는 안전장치가 놓치지 않고 정상적으로 감지한 사례로 해석해야 하며, 낮은 수치를 곧바로 QA 로직의 결함으로 해석해서는 안 된다.

5.3.1.5. 실측 테스트 결과

5.3.1.1~5.3.1.4에서 단계별로 측정한 값을 5.3.1의 정의에 따라 곱하면, 하나의 이상 케이스가 탐지부터 QA까지 전 단계를 정확하게 통과할 확률을 전체 정확도로 산출할 수 있다.

시나리오	탐지 정확도	분류 정확도	결정 정확도	액션 성공률	QA 통과율	전체
------	--------	--------	--------	--------	--------	----

						정확도
EC2 좀비	100%	100%(LLM)	100%	100%	100%	100%
EC2 오버프로비저닝	100%	80%(LLM)	80%	100%	100%	64.0%
Lambda 스로틀 재시도폭증	76.9%	88.9%(Rule Book)	100%	100%	100%	68.4%
S3 대량다운로드	92.3%	100%(Rule Book)	100%	100%	100%	92.3%
AutoScaling EDoS	73.9%	100%(8/8)	100%	100%	100%	73.9%

<표 11. 시나리오별 전체 파이프라인 테스트 결과>

5개 시나리오의 탐지 정확도와 전체 파이프라인 정확도를 종합하면 다음과 같다. 탐지 정확도는 시나리오별 표본 수(n)가 서로 달라(13~23) 단순평균과 표본 수를 반영한 가중평균을 함께 제시한다. 평균 탐지 정확도는 88.6%(단순평균), 86.7%(표본 수 가중평균)으로 나타났다. 평균 전체 정확도는 79.7%으로 나타났다.

시나리오	탐지 정확도	전체 정확도	n
EC2 좀비	100%	100%	13
EC2 오버프로비저닝	100%	64.0%	13
Lambda 스로틀 (재시도폭증)	76.9%	68.4%	13
S3 대량다운로드	92.3%	92.3%	13
AutoScaling EDoS	73.9%	73.9%	23

<표 12. 시나리오별 실측 정확도>

5.3.2. 비용

비용 추정치는 CloudWatch에서 조회한 실측 사용량에 AWS Pricing API로 조회한 단가를 곱해 산출한다. 따라서 Savings Plan, 예약 인스턴스 할인 등 실제 청구서에 반영되는 할인 요소는 포함되지 않으며, 본 절의 수치는 실제 청구액과 차이가 있을 수 있는 On-Demand 기준 추정치임을 밝힌다.

5.3.2.1. 비용 절감액

절감액은 "본 시스템을 적용하지 않았을 때의 비용"에서 "적용했을 때의 비용"을 뺀 값으로 정의하며, 산출 시점에 따라 예측치와 실측치로 나뉜다.

(1) 승인 요청 시점의 예측치

조치를 실행하기 전, decision_agent가 AWS 공식 단가표 또는 실측 비용 추세 (Throttle-ScaleIn-Block)를 근거로 계산한 값이다. 아래와 같이 리소스·액션 조합별로 서로 다른 로직으로 산출하며, 승인 화면에는 이 예측치만 존재한다.

리소스	액션	계산 함수	로직
EC2	Stop	_cost_based_full_removal_saving (fraction=1.0)	saving_rate=1.0, 절감액 = 평균 cost × 1.0 (전체 제거)
EC2	Stop+Schedule	_cost_based_full_removal_saving (fraction=0.5)	saving_rate=0.5(고정 가정치), 절감액 = 평균 cost × 0.5
EC2	Resize	_ec2_resize_saving()	AWS API로 현재 인스턴스 타입을 직접 조회 → 한 단계 아래 tier와의 단가 차이만큼 절감
Lambda	Throttle	_trend_based_partial_saving()	cost 시계열을 기준선(앞)/급증구간(뒤)으로 분리, 절감액 = max(0, 급증구간평균 - 기준선평균)
AutoScaling	Scale In	_trend_based_partial_saving()	위와 동일(급증분만 절감)
S3	Block	_trend_based_partial_saving()	위와 동일 — cost 시계열 자체가 bytes_downloaded(전송량) 기반 실제 데이터전송 비용을 반영

<표 13. 리소스별 액션에 따른 비용 절감액 계산 로직>

(2) 조치 이후의 실측 기반

조치 실행 후 충분한 시간이 지난 뒤, 미적용 시 비용(실측)에서 적용 시 비용(실측)을 실제로 빼서 계산한 값이다.

$$\text{절감액(실측)} = \text{시스템미적용 시 비용(예측)} - \text{시스템적용 시 비용(실측)}$$

5.3.2.2. 시스템 미적용 시 비용

본 시스템을 적용하지 않았다면 이상 상태가 그대로 유지되었을 것이라는 가정 하에, 조치 실행 직전의 실측 시간당 비용을 그대로 미래로 연장하여 추정한다.

$$\begin{aligned} & \text{시스템 미적용 시 비용} \\ & = \text{조치 직전 실측 시간당 비용}(\text{current_cost_usd/hr}) \times \text{경과 시간} \end{aligned}$$

여기서 경과 시간은 "해당 조치가 실행된 시점부터, 같은 리소스에 대한 다음 조치가 실행되는 시점까지"로 정의하며, 식으로 나타내면 다음과 같다. 후속 조치 없이 직전 조치가 계속 유지되는 경우에는 대시보드를 조회하는 시점까지의 경과 시간을 사용하므로, 이 경우 절감액은 조회할 때마다 동적으로 갱신된다.

$$\begin{aligned} & \text{경과 시간} = \text{다음 조치 실행 시각} - \text{해당 조치 실행 시각} \\ & (\text{후속 조치가 아직없는 경우: 경과 시간} = \text{현재 조회 시각} - \text{해당 조치 실행 시각}) \end{aligned}$$

5.3.2.3. 시스템 적용 시 비용

시스템 적용 시 비용 역시 두 시점에서 서로 다른 방식으로 산출되며, 두 값을 비교함으로써 예측의 정확도를 사후에 검증할 수 있다.

(1) 승인 요청 시점의 예측치

관리자가 승인 여부를 결정할 수 있도록, 조치를 실행하기 전 시점에 아래 계산 식으로 예측하여 웹 제어판에 제시한다. 이 값이 실측이 아닌 예측임을 화면에 명시한다.

$$\text{시스템 적용 시 비용 (예측)} = \text{미적용 시 비용} - \text{절감액(예측)}$$

(2) 조치 이후의 실측 재확인

조치 실행 후 일정 시간이 지난 뒤 QA 단계에서 CloudWatch를 재조회하여, 예측이 아닌 실측값으로 다시 계산한다.

시스템 적용 시 비용 (실측)

= 재조회한 실측 시간당 비용(after_cost_usd/hr) × 경과 시간

(1) 예측치와 (2) 실측치의 차이를 비교하면, 본 시스템의 절감액 예측이 실제 결과와 얼마나 부합하는지를 사후적으로 검증할 수 있다.

5.3.2.4. 실측 테스트 결과 (30일 기준)

절감액을 계산할 때는 30일을 고정 기준으로 설정하여 실측 테스트 결과를 정규화하였다. 실험마다 실제 측정 기간이 서로 달라 절대 비용을 그대로 비교하기에는 한계가 있으므로, 공통된 기간을 기준으로 환산할 필요가 있었다. 30일을 기준으로 정한 것은 AWS 비용이 월 단위로 청구되는 특성을 고려하여, 실제 운영에서 관리자가 체감하는 단위인 월간 절감액을 기준으로 비교하기 위함이다.

다만 이 30일 환산에는 예외가 있다. QA 검증 실패로 롤백된 조치는 실패로 기록되어 누적 절감액 산정 대상에서 제외하며, 롤백 이전까지 경과한 시간에 대해서도 절감액을 인정하지 않는다. 즉 조치가 최종적으로 실패한 것으로 확인되면, 그 조치가 일시적으로 유지되었던 기간의 비용 변화는 절감으로 계산하지 않는다.

EC2 Resize 실측 검증 사례를 보면, 절감액이 실측 사용량의 추세가 아니라 AWS 인스턴스 타입별 확정 단가 차이로 결정되므로, 조치가 완료된 즉시 실측 검증이 가능하며 예측과 실측이 정확히 일치하였다.

단계	값
예측 (_ec2_resize_saving())	target_type=t3.micro, estimated_saving_usd=0.0106 USD/hr
액션 실행 결과	status=success, new_instance_type=t3.micro
실측 검증 (describe_instances 재조회)	실제 타입 t3.micro 확인, 단가 0.0210→0.0104 USD/hr
절감액(실측)	0.0210 - 0.0104 = 0.0106 USD/hr
예측 오차	0.0000 USD/hr (예측과 실측이 정확히 일치)

표 14 . EC2 Resize 비용 실측 검증 사례

5.3.2.5. 실제 운영 규모 적용 시 비용

위에서 실측으로 확인한 인스턴스당 절감액(0.0106 USD/hr, t3.small→t3.micro 기준)을 30일로 환산하면 인스턴스당 약 \$7.63이다. 실제 운영 환경에서는 관리 대상 리소스 수에 비례하여 절감액이 늘어난다고 가정할 때(단가·낭비 비율이 동일하다는 전제) 아래와 같이 규모별 절감액을 추정할 수 있다.

오버프로비저닝된 EC2 인스턴스 수	30일 기준 예상 절감액
10대 (소규모 스타트업)	약 \$76.3
100대 (중견기업)	약 \$763
1,000대 (대기업)	약 \$7,632

<표 15. 인스턴스 수에 따른 예상 비용 절감액>

다만 이는 인스턴스당 절감액이 실험과 동일한 비율(t3.small→t3.micro 수준의 다운사이즈)로 선형 확장된다는 단순 가정에 기반한 것으로, 실제로는 조직마다 오버프로비저닝된 리소스의 비율과 인스턴스 유형 분포가 다르므로 참고용 추정치로만 활용해야 한다.

5.3.3. 실행 소요 시간

전체 소요 시간은 6단계(탐지→분류→결정→조치→검증→기록)를 순차 실행한 값의 합이다.

$$\text{전체실행소요시간} = \text{detection} + \text{classification} + \text{decision} + \text{action} + \text{qa} + \text{logging}$$

단계별 소요 시간과 변동 요인은 다음과 같다 .

- Detection : 이미 조회된 CloudWatch 지표로 z-score/IForest 점수만 계산하기에, 빠르고 변동이 거의 없다.

$$\text{Detection소요시간} = Z_{score} \text{ 계산 시간} + \text{IForest 점수 계산 시간}$$

- Classification

- Rule Book에 매칭되는 경우

$$\text{Classification소요시간} = \text{규칙 매칭 시간}$$

- Rule Book에 매칭되지 않는 경우 (LLM 호출) : 네트워크 왕복 시간과 모델 추론 시간이 추가돼 수 초~수십 초가 소요된다. 따라서 동일한 판단이 반복되어 Rule Book으로 승격되면, 이후에는 LLM 호출 자체가 생략되므로 Classification 소요 시간이 크게 단축될 것으로 기대된다.

$$Classification\text{소요시간} = \text{규칙 매칭 실패 시간} + LLM\ \text{호출 시간}$$

- Decision

- Rule Book에 매칭되는 경우

$$Decision\text{소요시간} = \text{규칙 매칭 시간}$$

- Rule Book에 매칭되지 않는 경우 : 네트워크 왕복 시간과 모델 추론 시간이 추가돼 수 초~수십 초가 소요된다.

$$Decision\text{소요시간} = \text{규칙 매칭 실패 시간} + LLM\ \text{호출 시간}$$

- Action : 액션 종류마다 API 호출 시간과 상태전이 대기 유무에 차이가 있다 . 단순 API 1회(Block)는 1초 미만, 인스턴스 상태 전이 동반(Resize/Scale In, WAF 연동 겹치는 경우)은 수십 초까지도 소요된다 .

$$Action\text{소요시간} = AWS\ API\ \text{호출 시간} + AWS\ \text{상태전이 대기 시간}$$

- QA :

- No Action인 경우 (QA를 스킵)

$$QA\text{소요시간} = \text{스킵시간} \ (\approx 0\ s)$$

- 실제 조치가 실행된 경우 : 본 시스템은 리소스 타입과 무관하게 지표 갱신 주기를 5분(300s)으로 통일하였다 . 따라서 조치 직후 300s를 고정 대기한 뒤 갱신된 메트릭·비용·가용성으로 SLA를 판단한다.

$$\begin{aligned} QA\text{소요시간} \\ = \text{메트릭 갱신 대기 시간}(300s) + \text{메트릭 재조회 시간} + SLA\ \text{판단 시간} \end{aligned}$$

- Logging : 로컬 저장만 수행하므로 빠르고 변동이 거의 없다.

$$\text{Logging 소요시간} = \text{DB 삽입 시간} + \text{JSONL 로그 추가 시간}$$

액션까지 도달한 실측 데이터가 있는 3개 시나리오(S3, EC2 오버프로비저닝, Lambda 스로틀)를 대상으로 단계별 소요시간을 측정하였다.

단계	S3 (n=3)	EC2 오버프로비저닝 (전체 n=13, 액션도달 n=5)	Lambda 스로틀 (n=8) (전체 n=13, 액션도달 n=8)
탐지 (Detection)	0.026초	0.106초	0.343초
분류 (Classification)	0.004초	16.942초	0.000초*
결정 (Decision)	0.016초	4.463초	0.002초
조치 (Action)	0.308초	17.272초	0.888초
검증 (QA)	300.140초	301.131초	300.927초
기록 (Logging)	0.007초	2.042초	0.090초
전체 실행시간 평균	300.63초	평균 134.94초**	평균 187.19초**

<표 16. 3개의 시나리오에 대한 단계별 소요시간 평균>

* Rule Book 규칙에 매칭되어 LLM 호출이 생략된 경우의 값이다.

** 액션까지 도달하지 않은(NoAction) 표본이 섞인 전체 표본(n=13) 기준 평균이라 낮게 나타난다.

MTTD(Mean Time To Detect, 평균 탐지 소요시간)는 이상 징후가 발생한 시점부터 탐지 에이전트가 이를 인지하기까지 걸리는 평균 시간을 의미하며, 본 시스템에서는 탐지(Detection) 단계 자체의 평균 소요시간으로 측정하였다. MTTR(Mean Time To Resolve, 평균 복구 소요시간)는 이상이 탐지된 시점부터 조치가 실행되고 그 결과가 검증(QA)까지 완료되어 실제로 문제가 해소되었다고 확인되는 시점까지 걸리는 평균 시간으로 정의하였다. 즉 탐지·분류·결정·조치·검증 5단계를 모두 포함한다.

MTTR이 300초를 크게 웃도는 것은 검증(QA) 단계의 고정 대기 시간(약 300초)이 포함되었기 때문이다. 이는 리소스 타입과 무관하게 지표 갱신 주기를 5분으로 통일한 설계에 따른 것으로, 조치 직후에는 CloudWatch 지표에 변경 사항이 충분히 반영되지 않을 수 있어 5분의 대기 후 갱신된 값을 기준으로 검증하도록 하였다. 한편 MTTR의 평균(313.7초)과 중앙값(302.3초)이 차이 나는 이유는, EC2 오버프로비저닝의 조치 1건에서 분류(Classification) 단계의 LLM 호출이 지연되어 21초 이상 소요된 이

상치가 포함되었기 때문이다. 대부분의 경우 QA 대기 시간을 제외한 실제 판단·조치 시간은 1~2초 내외였으며, 해당 LLM 호출 지연이 평균 실행 시간을 증가시킨 것으로 나타났다.

지표	값
MTTD	0.204초 (n=29, 3개 시나리오 통합)
MTTR	평균 313.746초 / 중앙값 302.317초 (n=16)

<표 17 . MTTD / MTTR>

5.4. 시스템 성능 통계적 분석 (Clopper-Pearson 신뢰구간)

소표본 반복실행 결과는 점추정치와 함께 95% Clopper-Pearson 신뢰구간을 함께 보고하였다(계산 방식은 3.1.7절 참고). 대표 결과는 다음과 같다.

시나리오	n(정상/이상)	정확도 [95% CI]	재현율 [95% CI]	위양성률(FPR) [95% CI]
EC2 좀비	8 / 5	100.0% [75.3%, 100%]	100.0% [47.8%, 100%]	0.0% [0%, 36.9%]
EC2 오버프로비저닝	8 / 5	100.0% [75.3%, 100%]	100.0% [47.8%, 100%]	0.0% [0%, 36.9%]
Lambda 스로틀 재 시도폭증	8 / 5	76.9% [46.2%, 95.0%]	100.0% [47.8%, 100%]	37.5% [8.5%, 75.5%]*
S3 대량다운로드	8 / 5	92.3% [64.0%, 99.8%]	100.0% [47.8%, 100%]	12.5% [0.3%, 52.7%]
AutoScaling EDoS	8 / 15	73.9% [51.6%, 89.8%]	66.7% [38.4%, 88.2%]	12.5% [0.3%, 52.7%]

<표 18. 시나리오별 시스템 성능 통계적 분석>

EC2 좀비·오버프로비저닝의 Recall 100%는 95% Clopper-Pearson 신뢰구간이 [75.3%, 100%]로 나타나, 점추정치와 달리 표본 수가 적어 불확실성이 존재함을 확인할 수 있다. S3·Lambda(구버전) 역시 표본 수가 적어 신뢰구간이 넓게 나타났으며, Lambda 스로틀의 FPR 37.5%도 [8.5%, 75.5%]의 넓은 구간을 보였다. 한편 AutoScaling EDoS는 가장 많은 표본(n=23)을 확보하여 Recall 66.7%의 신뢰구간이 상대적으로 좁게 나타났으며, v5(n=10, Recall 80%)보다 점추정치는 낮아졌지만 표본

확대를 통해 보다 안정적인 성능 추정이 가능해졌다. 따라서 본 실험에서는 Recall 100%와 같은 점추정치만으로 성능을 판단하기보다, 표본 크기와 신뢰구간을 함께 고려할 필요가 있다.

5.5. 버그 발견 및 수정 이력

개발 과정에서 Isolation Forest 기반 이상 탐지 결과가 Rule Book 과 정상적으로 연계되지 않는 문제가 발견되었다. 특히 Isolation Forest 만으로 이상이 탐지된 경우 triggered_metrics 가 비어 있어 기존 규칙의 매칭 조건을 충족하지 못하는 문제가 발생하였다. 이를 해결하기 위해 탐지 결과에서 급증한 지표를 자동으로 추출하는 로직을 추가하고, 해당 정보를 Rule Book 매칭 과정에 활용하도록 수정하였다. 이를 통해 탐지 방식에 관계없이 이상 탐지 결과가 이후 분류 및 규칙 기반 처리 단계로 일관되게 전달되도록 개선하였다.

Rule Book 의 규칙 승격 과정에서는 반복적인 LLM 판단이 일정 조건을 만족하면 별도의 검증 없이 자동으로 규칙에 반영될 수 있다는 안전성 문제가 발견되었다. 잘못된 판단이 규칙으로 승격될 경우 이후 동일한 상황에서 잘못된 조치가 반복될 가능성이 있기 때문에, 규칙 승격 방식을 관리자 승인 기반으로 변경하였다. 반복 판단과 QA 검증을 통과한 패턴은 우선 승인 대기 큐에 등록하고, 관리자가 검토한 후 승인한 경우에만 Rule Book 에 반영되도록 수정하여 자동화 과정에서 발생할 수 있는 판단 오류의 확산을 방지하였다.

관리자 웹 시스템에서는 PostgreSQL 연결 실패가 전체 관리자 API 의 기동 실패로 이어지는 문제가 확인되었다. 기존 구조에서는 데이터베이스 연결 여부가 API 의 정상적인 실행에 직접적인 영향을 주었으나, 이를 수정하여 PostgreSQL 이 일시적으로 연결되지 않는 상황에서도 데이터베이스 의존성이 없는 기본 API 기능은 동작할 수 있도록 방어 로직을 추가하였다. 이를 통해 외부 저장소 장애가 관리자 시스템 전체의 가용성 저하로 이어지는 범위를 줄였다.

탐지 양상불 구조를 재검토하는 과정에서는, Isolation Forest-Z-score-EC2 유희-Lambda 에러 재시도-Lambda Throttle 재시도라는 다섯 개의 독립된 판정 경로가 하나의 OR 게이트로 병렬 결합되어 있어 서로 다른 판정 철학(다변량 모델 대 개별 지표 절대임계값)이 뒤섞여 있다는 문제가 확인되었다. 학습된 Isolation Forest 트리 구조를 직접 분석한 결과, 100 개 트리 중 17 개만 루트 노드에서, 46 개만 상위 3 레벨 이내에서 리소스 유형을 구분하는 것으로 나타났다. 이 때문에 EC2 유희처럼 특정 지표 값이 다른 리소스 유형의 '해당 없음'(0 으로 채워진 값)과

구조적으로 겹치는 경우 모델이 안정적으로 구분하지 못하는 반면, Lambda 의 에러율·스로틀률처럼 전역적으로 희귀한 값 대역을 갖는 지표는 모델이 안정적으로 분리해 낼 수 있음을 실측으로 확인하였다. 이에 따라 EC2 유휴 판정만 결정론적 절대임계값 게이트로 남기고, Lambda 의 에러·스로틀 재시도 판정은 독립 게이트를 제거해 Isolation Forest 의 다변량 입력 피쳐로 통합하는 방향으로 앙상블 구조를 재설계하였다.

AutoScaling EDoS 시나리오의 실제 AWS 연동 실험에서는 같은 날 세 차례의 반복 실측에서 모두 Recall 0%가 나타나는 문제가 확인되었다. 원인을 추적한 결과, 세 가지 요인이 복합적으로 작용한 것으로 분석되었다. 첫째, 트래픽이 급증하는 시점의 값이 판정 창의 평균과 표준편차를 함께 증가시켜 Z-score 가 해당 급증을 정상 범위로 판단하는 masking effect 가 발생하였다. 둘째, 콜드스타트 안정화를 위해 도입한 Isolation Forest 의 버퍼 동결 옵션이 계속 활성화되어 있어, 모델이 실측 데이터를 학습하지 못하고 초기 시드 상태에 고정되어 있었다. 셋째, 앞선 두 문제를 해결한 이후에도 재현 실험에서 탐지가 실패하였는데, 이는 desired_capacity 가 실험 규모(1~4 대)에서 가질 수 있는 값의 종류가 매우 제한적인 이산형 지표이므로 통계적 방법이 학습 및 이상 판정에 활용할 수 있는 정보가 부족했기 때문인 것으로 확인되었다. 동일한 무임계값 방식으로 연속값에 가까운 누적 지표인 S3 다운로드량에서는 Recall 100%가 나타났다는 점도 이러한 판단을 뒷받침하였다. 이에 EDoS 의 근본 원인을 인스턴스 수와 같은 결과 지표가 아니라 트래픽과 같은 원인 지표에서 탐지해야 한다고 판단하고, 실험을 전면 재설계하였다. desired_capacity 를 1 대로 고정하고 ALB 와 리소스별 전용 Target Group 을 구성한 뒤, 각 인스턴스에 최소한의 HTTP 서버를 실행하였다. 이후 정상군에서는 낮고 일정한 요청을, 이상군에서는 기준치 대비 급증한 요청을 실제로 발생시키고, ALB 의 누적 요청 수(RequestCount)를 탐지 지표로 활용하도록 변경하였다. 재설계 과정에서는 추가적인 구현 및 환경 설정 문제도 확인하였다. CloudWatch 에서 인스턴스 지표 수집이 기본적으로 비활성화되어 지표가 모두 0 으로 조회되는 문제, ALB ARN 과 Target Group ARN 의 슬라이싱 방식 차이로 인해 RequestCount 조회가 항상 0 을 반환하던 문제, 그리고 보안 그룹의 포트 허용 범위가 실험 규모에 비해 제한적으로 설정되어 일부 트래픽이 ALB 에 도달하지 못했음에도 외부에서는 단순히 트래픽이 낮은 정상 상태로만 관측되는 문제가 발생하였다. 해당 문제들을 순차적으로 수정한 후, 실제 트래픽을 기반으로 한 지표에서 유의미한 탐지 결과를 확보할 수 있었다.

AutoScaling EDoS 시나리오의 WAF Rate-based Rule 연동 과정에서는, 신규 생성된 Web ACL 을 리소스에 연결하는 과정에서 AWS 내부 전파 지연으로 인해

반복적으로 연결이 실패하는 문제가 확인되었다. 최초에는 2 초, 4 초 간격으로 재시도하도록 처리하였으나 실측 결과 8 회 모두 동일하게 실패하였으며, 별도의 재현 실험을 통해 실제 전파 지연이 초 단위가 아니라 30 초 내외로 훨씬 길다는 것을 확인하였다. 이에 재시도 간격을 30 초로 늘리고 재시도 횟수도 함께 늘려 반영하였으며, 수정 이후 재현 실험에서 1 회 재시도 만에 정상적으로 연결되는 것을 확인하였다. 이를 통해 신규 생성된 AWS 리소스를 즉시 참조할 때 발생할 수 있는 전파 지연 문제에 대응할 수 있도록 하였다.

Isolation Forest 모델의 동시 실행 안정성을 점검하는 과정에서는, 모델과 학습 버퍼 파일에 동시성 제어가 없어 여러 스레드가 동시에 접근할 경우 문제가 발생할 수 있음이 확인되었다. 실제로 동일한 리소스를 동일한 지표로 연달아 두 번 판정하였을 때, 그 사이 다른 스레드가 모델을 갱신하면서 두 번째 판정 결과가 True 에서 False 로 뒤바뀌는 사례가 실측으로 확인되어, 모델 로드·학습 함수 전체를 임계구역으로 묶어 수정하였다.

개발 과정에서 EC2 좀비 리소스에 대한 조치가 정상적으로 실행되었음에도 QA 검증 단계에서 실패로 판정되어 롤백되는 문제가 발견되었다. 원인을 분석한 결과, QA Agent 가 조치 이후의 최신 지표가 아닌 Detection 단계에서 수집한 조치 전 raw_metrics 를 그대로 재사용하여 검증하고 있었으며, 중간 단계에서도 해당 지표가 갱신되지 않는 것이 원인이었다. 이를 해결하기 위해 조치 실행 후 5 분간 대기한 뒤 CloudWatch 에서 최신 지표를 다시 조회하여 raw_metrics 를 갱신하는 로직을 추가하고, 조회에 실패한 경우에는 기존 데이터를 사용하도록 보완하였다. 이를 통해 QA 가 조치 이후의 리소스 상태를 기준으로 검증할 수 있도록 개선하였다.

개발 과정에서 EC2 Resize 조치의 예상 절감액을 계산할 때 실제 인스턴스 타입이 아닌 평균 비용을 단가표와 비교하여 인스턴스 타입을 역추정함으로써 부정확한 결과가 발생할 가능성을 확인하였다. 기존 방식은 서로 다른 인스턴스 타입의 비용이 유사한 경우 실제 타입과 다르게 추정될 수 있다는 한계가 있었다. 이를 해결하기 위해 AWS describe_instances()를 통해 실제 인스턴스 타입을 직접 조회하여 절감액 계산에 우선 사용하고, 조회에 실패하거나 단가표에 등록되지 않은 타입인 경우에만 기존 비용 기반 역추정 방식을 사용하도록 수정하였다. 이를 통해 실제 리소스 정보를 기반으로 절감액을 보다 정확하게 산정할 수 있도록 개선하였다. 이 개선은 산학협력 멘토의 자문 의견을 반영한 것이다.

6. 결론 및 향후 연구 방향

본 연구는 클라우드 비용 이상 징후를 탐지하는 데 그치지 않고, 분류-결정-실행-검증-기록으로 이어지는 자율 복구 파이프라인을 LangGraph 기반 에이전트 구조로 실제 AWS 환경에 구현하였다. Isolation Forest와 Z-score, EC2 유휴 상태 절대임계값을 결합한 OR 앙상블 기반 이상 탐지와 Rule Book 및 LLM을 활용한 분류, 결정론적으로 산출한 절감률과 관리자 조정이 가능한 우선순위 가중치에 기반한 조치 선택, interrupt와 체크포인트를 활용한 승인 게이트, QA 및 롤백을 통한 안전장치, 그리고 실시간 상태 추적·Grafana·Slack을 통한 관제를 하나의 파이프라인으로 통합하였다. 이를 통해 단순히 비용 이상을 탐지하고 알림을 제공하는 것을 넘어, 탐지된 이상에 대해 조치의 필요성을 판단하고 실행 및 결과 검증까지 수행하는 자동화 구조를 구현했다는 점에서 의미가 있다.

실제 AWS 환경에서 5개 시나리오(EC2 좀비·EC2 오버프로비저닝·Lambda 스로틀 재시도폭증·S3 대량다운로드·AutoScaling EDoS)에 대해 반복 실험을 수행한 결과, 평균 탐지 정확도는 88.6%(단순평균)/86.7%(표본 수 가중평균), Recall은 85.7%(95% CI [69.7%, 95.2%])로 나타났다. 또한 탐지 파라미터를 조정하면서 재현율과 Joint Catch Rate를 개선할 수 있음을 확인하였다. 이러한 결과는 제안한 탐지 구조가 다양한 클라우드 자원에서 비용 및 운영 이상을 식별하는 데 활용될 수 있음을 보여준다. 다만 반복 실험의 표본 수가 시나리오별로 13~23개 수준으로 제한되어 있어 해당 결과를 일반화하기에는 한계가 있으며, 장기간의 다양한 조건에서 추가적인 검증이 필요하다.

특히 본 연구에서는 자동화에 따른 오작동 위험을 고려하여 단순한 자동 실행보다 관리자의 통제권을 유지하는 방향으로 시스템을 설계하였다. 위험도가 낮은 조치는 자동으로 수행하되, 중간 이상 위험도의 조치는 Slack을 통한 알림과 승인 게이트를 거치도록 구성하였으며, 체크포인트를 통해 승인 대기 중인 그래프의 상태를 보존하고 승인 이후 동일한 실행 상태에서 작업을 재개하도록 구현하였다. 또한 조치 이후 QA를 수행하여 CPU 사용률, 비용 증가 여부, 조치 성공 여부를 검증하고, 실패 시 롤백 및 재시도를 수행하도록 구성하였다. 이를 통해 비용 절감 효과뿐만 아니라 클라우드 자원에 대한 조치의 안전성과 관리 가능성을 함께 고려한 자율 복구 구조를 구현하였다.

한편 실제 환경에서의 검증에는 몇 가지 한계가 존재하였다. 계정 단위 vCPU 한도 등의 AWS 환경 제약으로 인해 Auto Scaling 및 S3 시나리오의 대규모 실 AWS 반복 검증이 제한적이었으며, 일부 시나리오는 축소된 규모 또는 제한적인

조건에서 검증하였다. 또한 반복 실험의 표본 수가 충분하지 않아 탐지 성능의 통계적 신뢰성을 확보하는 데 한계가 있었고, LLM 기반 분류·결정 판단은 동일한 입력에 대해서도 호출 시점에 따라 다른 결과가 나타날 수 있어, 판단의 일관성 측면에서 근본적인 한계가 존재하였다. 향후에는 리소스 유형별 장기간 반복 실험을 통해 탐지 성능의 신뢰구간과 안정성을 추가로 검증하고, Auto Scaling·S3·RDS 등 다양한 리소스 유형과 실제 운영 환경을 대상으로 검증 범위를 확대할 필요가 있다. 또한 Rule Book 승격 기준을 정량화하여 반복적인 LLM 판단을 보다 안정적으로 규칙화하고, 위험도 산정 및 승인 우선순위 로직을 고도화하여 관리자의 승인 부담을 줄이는 방향으로 시스템을 발전시킬 수 있다.

또한 운영 환경에서 수집되는 실측 데이터의 안정성과 탐지 결과의 신뢰성을 충분히 검증한 후, 현재 도입된 자기참조 재학습 구조의 안정성과 탐지 결과의 신뢰성을 운영 환경에서 충분히 검증한 후, 장기 운영 조건으로 확대 적용할 수 있다. 다만 이 재학습 구조는 동일 프로세스 내 여러 스레드가 모델 파일에 동시 접근할 때 발생하는 불일치 문제를 임계구역 설정으로 해결하였을 뿐, 서로 다른 프로세스가 동시에 접근하는 경우에 대한 보호는 아직 마련되어 있지 않아 향후 보완이 필요하다. 기업 및 서비스별로 클라우드 리소스의 구성과 사용 패턴, 트래픽 패턴 등이 서로 다르기 때문에 동일한 지표라 하더라도 정상 범위가 달라질 수 있다. 따라서 각 운영 환경에서 축적되는 실측 데이터를 지속적으로 학습에 반영함으로써 mock 데이터만으로는 반영하기 어려운 기업별 정상 사용 패턴에 맞춰 이상 탐지를 정교화하는 방향으로 발전시킬 수 있다.

다만 그리드 서치는 합성 평가 데이터와 정상 데이터 150 개를 일괄 학습한 실험 모델을 기반으로 수행되어, 실제 운영 환경과 데이터 및 학습 조건에 차이가 있다. 따라서 선정된 파라미터는 향후 충분한 실측 데이터를 확보한 후 실제 운영 환경에서 재검증할 필요가 있다.

향후에는 과거 정상 데이터를 기반으로 한 정상 범위와 현재 윈도우의 상대적 이상 점수를 함께 비교하는 방식으로 Isolation Forest의 판정 구조를 개선할 필요가 있다. 현재 이상 점수는 하나의 윈도우를 구성하는 30 개 시점 내부에서 min-max 정규화하므로, 절대적인 이상 수준보다 윈도우 내 상대적 차이에 영향을 받는 한계가 있다. 이로 인해 전체 시점이 정상인 경우에도 상대적으로 높은 점수가 이상으로 판정될 수 있으며, 반대로 전체 시점이 비정상적인 수준임에도 실제 이상 구간의 점수가 낮게 정규화되는 현상이 일어날 수 있다. Auto Scaling 용량이 1에서 3으로 급증한 이상에서 윈도우 전체가 비정상적이었음에도 잡아내지 못한 사례가 있다.

7. 구성원별 역할 및 개발 일정

학번	이름	주요 담당 영역
202355540	박소영	- 이상 탐지 모델 구현(Isolation Forest/Z-score) - 로깅·DB 적재 - Grafana 대시보드 구축 - EC2 좀비, Lambda 재시도 폭증 반복실험
202355514	강지원	- Rule Book 설계 및 자동 승격 로직 구현 - QA 판정 및 롤백 조건 구현 - WAF 등 인바운드 보안 조치 핸들러 구현 - Autoscaling EDoS 반복실험
202355594	허소영	- 규칙·LLM 기반 판단 로직 구현 - 리소스별 조치 실행 구현 - 관리자 웹제어판 및 Slack 알림 구현 - EC2 오버프로비저닝, S3 다운로드 폭증, EDoS 반복실험

단계	주요 내용	5월	6월	7월	8월	9월
설계	기능 구체화					
	DB 설계					
개발	Detection/Classification/Logging Agent 구현					
	Decision/Action/QA Agent 구현					
	Rule Book / 화이트리스트 / LLM 자동 판단 연동					
	IsolationForest 통합 모델 구현·개선					
	백엔드 개발					
	프론트 개발					
통합	통합테스트					
실측 검증	S3/EC2/Lambda/AutoScaling 실제 AWS 반복실험 및 통계 검증					

8. 참고 문헌

8.1. 참고 문헌 목록

- [1] Flexera, "2026 State of the Cloud Report," 2026. [Online]. Available: <https://info.flexera.com/CM-REPORT-State-of-the-Cloud> (accessed: 2026-09-13)
- [2] Flexera, "New Flexera Report Finds that 84% of Organizations Struggle to Manage Cloud Spend," Press Release, 2026. [Online]. Available: <https://www.flexera.com/about-us/press-center/new-flexera-report-finds-84-percent-of-organizations-struggle-to-manage-cloud-spend> (accessed: 2026-09-13)
- [3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. on Data Mining (ICDM), Pisa, Italy, 2008, pp. 413-422.
- [4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 4765-4774.
- [5] C. J. Clopper and E. S. Pearson, "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial," Biometrika, vol. 26, no. 4, pp. 404-413, 1934.
- [6] LangChain, Inc. LangGraph Documentation - Persistence (StateGraph, Checkpointer). [Online]. Available: <https://langchain-ai.github.io/langgraph/concepts/persistence/> (accessed 2026-09-12).
- [7] LangChain, Inc. LangGraph Documentation - Human-in-the-loop (interrupt, Command). [Online]. Available: https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/ (accessed 2026-09-12).
- [8] Amazon Web Services. Boto3 Documentation. [Online]. Available: <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html> (accessed 2026-09-10).
- [9] Amazon Web Services. AWS Cost Explorer / AWS Budgets User Guide. [Online]. Available: <https://docs.aws.amazon.com/cost-management/> (accessed 2026-09-10).
- [10] FinOps Foundation. The FinOps Framework. [Online]. Available: <https://www.finops.org/framework/> (accessed 2026-09-10).

[11] scikit-learn developers. sklearn.ensemble.IsolationForest. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html> (accessed 2026-09-10).

[12] Amazon Web Services. AWS WAF Developer Guide - Rate-based rule statement. [Online]. Available: <https://docs.aws.amazon.com/waf/latest/developerguide/waf-rule-statement-type-rate-based.html> (accessed 2026-09-12).

[13] S. Amershi et al., "Guidelines for Human-AI Interaction," in Proc. 2019 CHI Conf. on Human Factors in Computing Systems, 2019, pp. 1-13.